

Платформа. Ошибки НЕВЫЖИВШИХ

Основано на реальных событиях

Павел Богданов

Менеджер продукта,
Deckhouse Kubernetes Platform
Community Edition



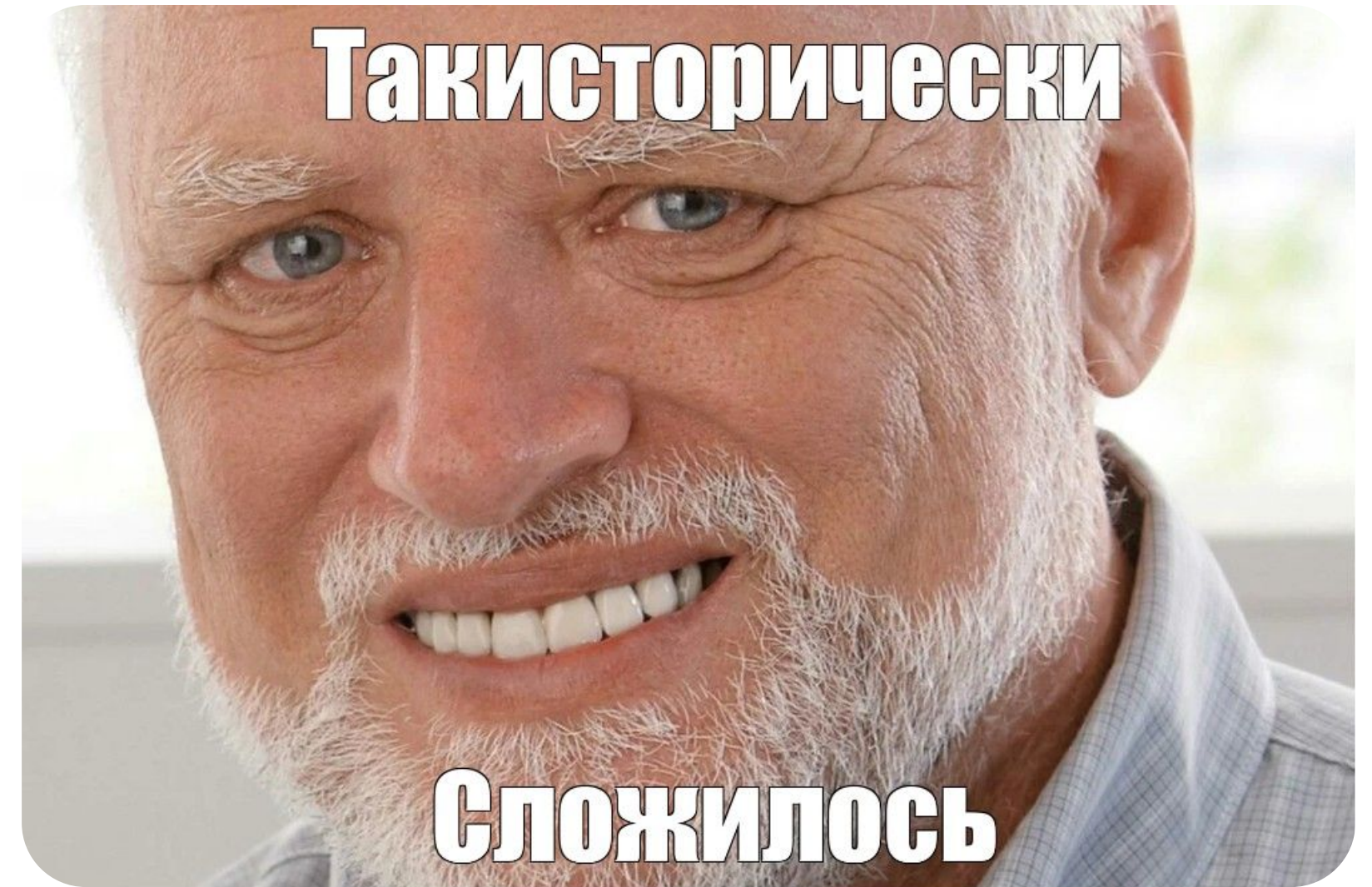
Так исторически сложилось!

Плюсы

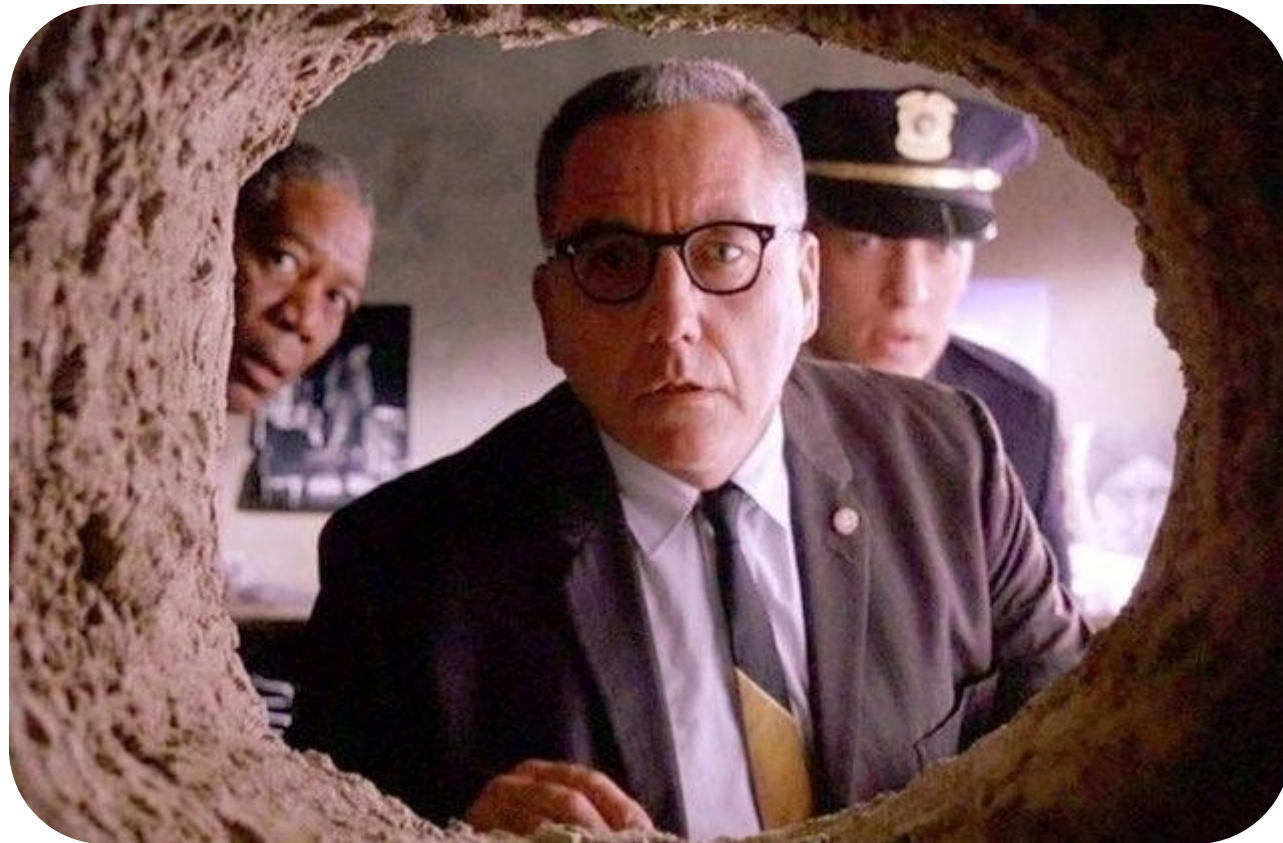
- + Ошибка признаётся
- + Ответственность снимается, можно выдохнуть

Минусы

- Точная причина неизвестна
- Всё может повториться



Любая абстракция в некоторой степени дырявая



(с) Закон дырявых абстракций

Абстракция — упрощение чего-то
значительно более сложного,
происходящего «под капотом».

ТСР абстракция над IP

Абстракции повсюду

Абстракции повсюду



Процесс работы

Абстракции повсюду



Процесс работы



Организационная структура

Абстракции повсюду



Процесс работы



Организационная структура



Соглашение или ТЗ

Абстракции повсюду



Процесс работы



Организационная структура



Соглашение или ТЗ



Технология, система

Абстракции повсюду



Процесс работы



Организационная структура



Соглашение или ТЗ



Технология, система



КОД!

Абстракции могут
строиться
на абстракциях,
поэтому...

**Абстракции могут
строиться
на абстракциях,
поэтому...**

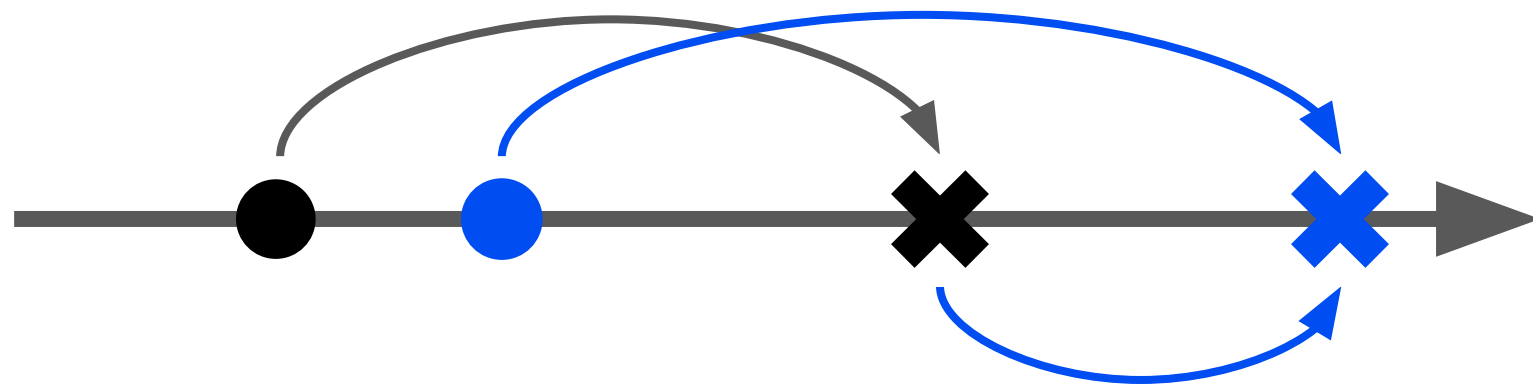
- ✓ Проблема и её причина лежат на разных уровнях

**Абстракции могут
строиться
на абстракциях,
поэтому...**

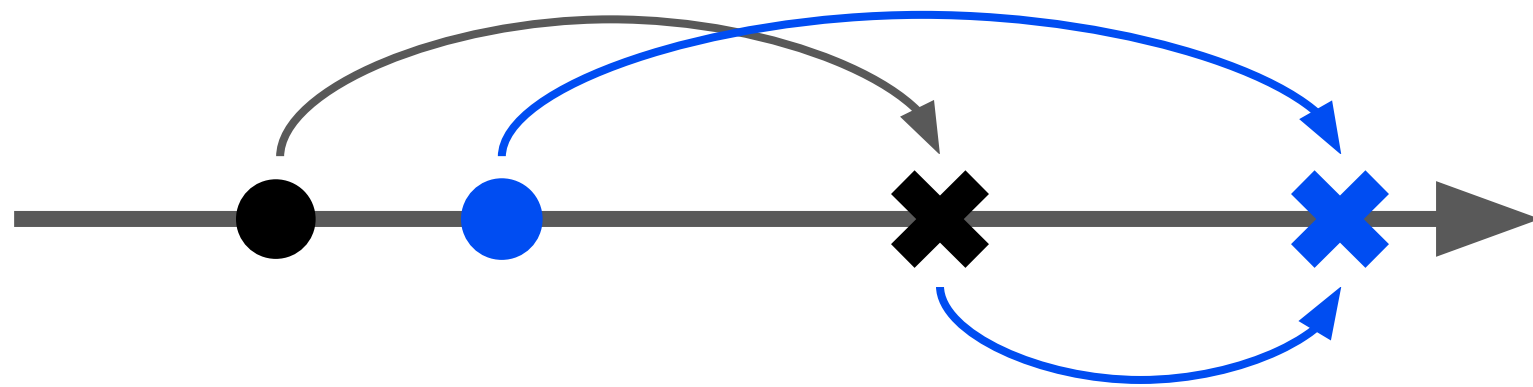
- ✓ Проблема и её причина лежат на разных уровнях
- ✓ Бесполезно смотреть на результат, надо смотреть на решения и контекст, которые к этому привели

Абстракции могут строиться на абстракциях, поэтому...

- ✓ Проблема и её причина лежат на разных уровнях
- ✓ Бесполезно смотреть на результат, надо смотреть на решения и контекст, которые к этому привели

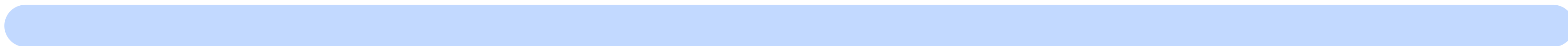


Абстракции могут строиться на абстракциях, поэтому...



- ✓ Проблема и её причина лежат на разных уровнях
- ✓ Бесполезно смотреть на результат, надо смотреть на решения и контекст, которые к этому привели
- ✓ Важно знать, **что именно каждая абстракция скрывает**

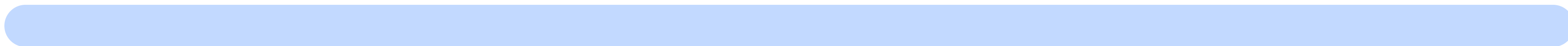
Впереди 7 историй про внутренние платформы



- №1 Давай сделаем это вместе
- №2 Требования собрать нельзя усреднить
- №3 Любую проблему можно решить, добавив ещё один слой абстракции
- №4 Нажмите кнопку...
- №5 А что потом?
- №6 Груз ответственности
- №7 Долбанёт? Вероятно, нет!

История № 1.

Давай сделаем это вместе



Встречаются СТО и СЮ.

СТО: У нас сейчас в фокусе ТТМ и оптимизация затрат. И, короче, нужна инфраструктурная платформа.

СЮ: У нас то же самое!
Ок, давай сделаем.



Совпадают ли их цели?
Почему?

Да, но нет

СТО отвечает за разработку – ТТМ >> Затраты

СІО – за инфраструктуру – Затраты >> ТТМ

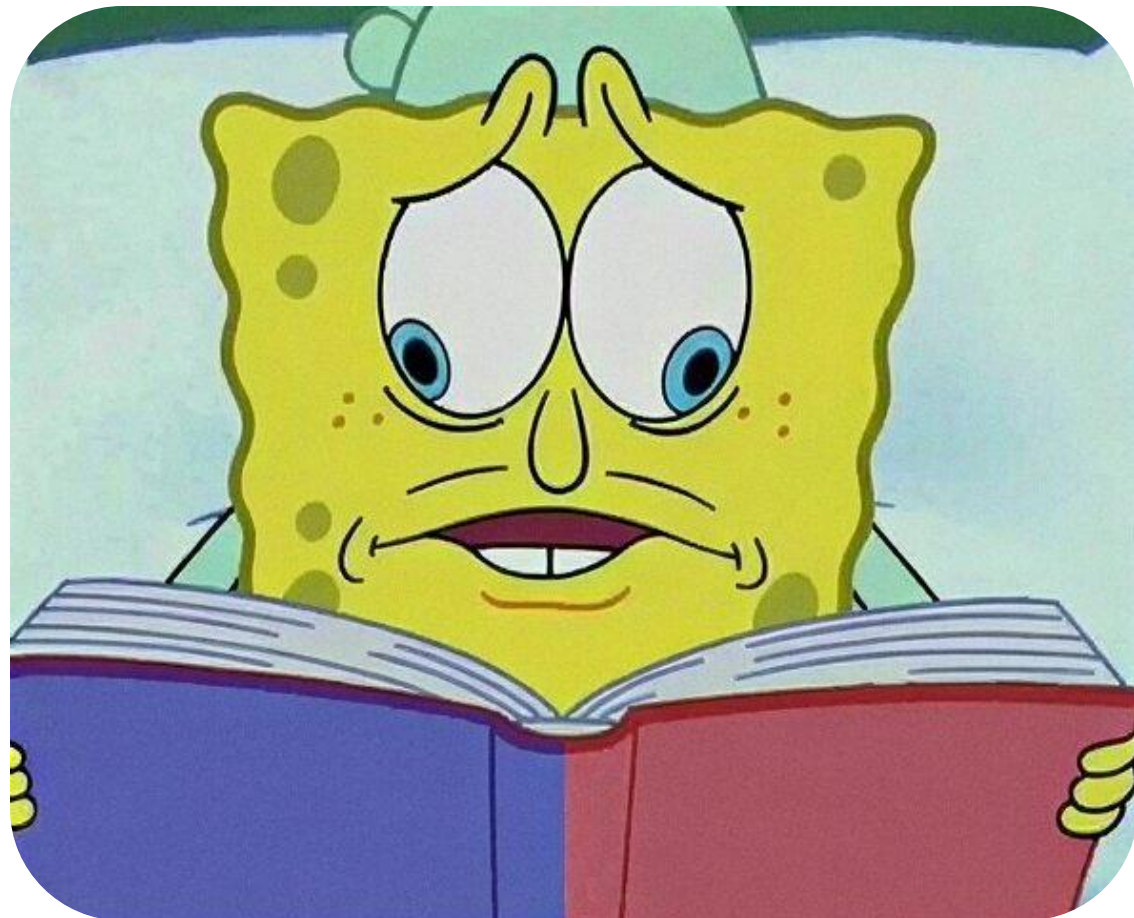
В массивах одинаковые элементы, но приоритет разный.

В стремлении договориться ребята абстрагировали приоритет отдельных метрик.

**СОВПАДЕНИЕ?
НЕ ДУМАЮ.**

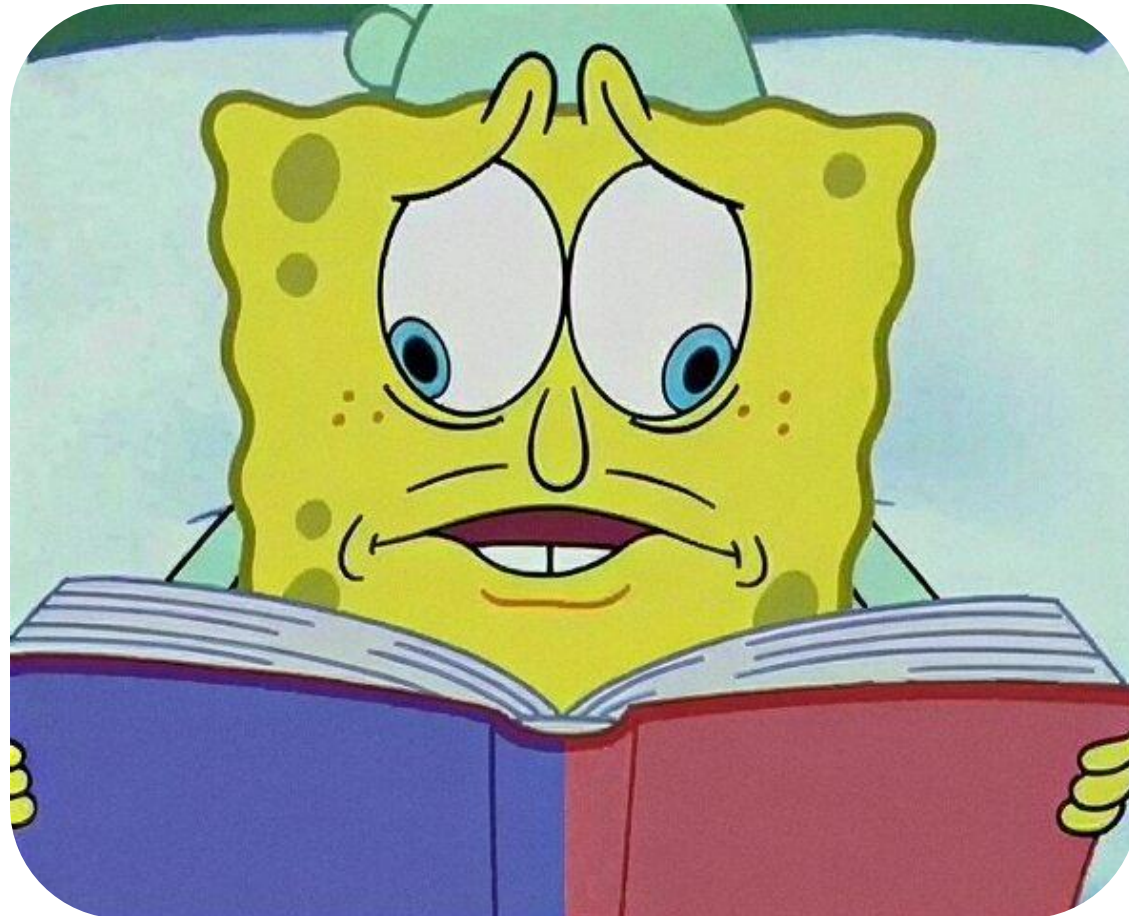


У одной системы
не может быть
2 главных целей



Борьба систем убьёт обе метрики

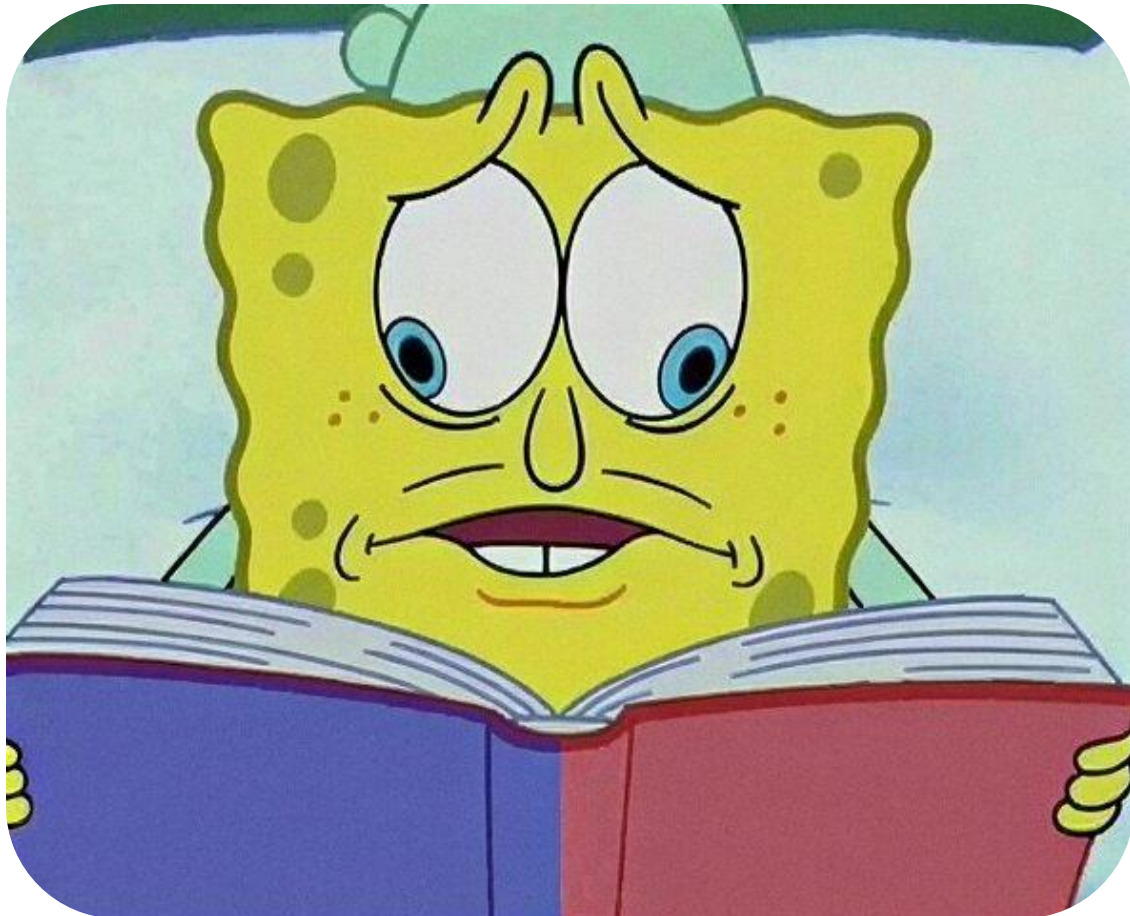
**У одной системы
не может быть
2 главных целей**



Борьба систем убьёт обе метрики

- Система распадётся на 2 подсистемы

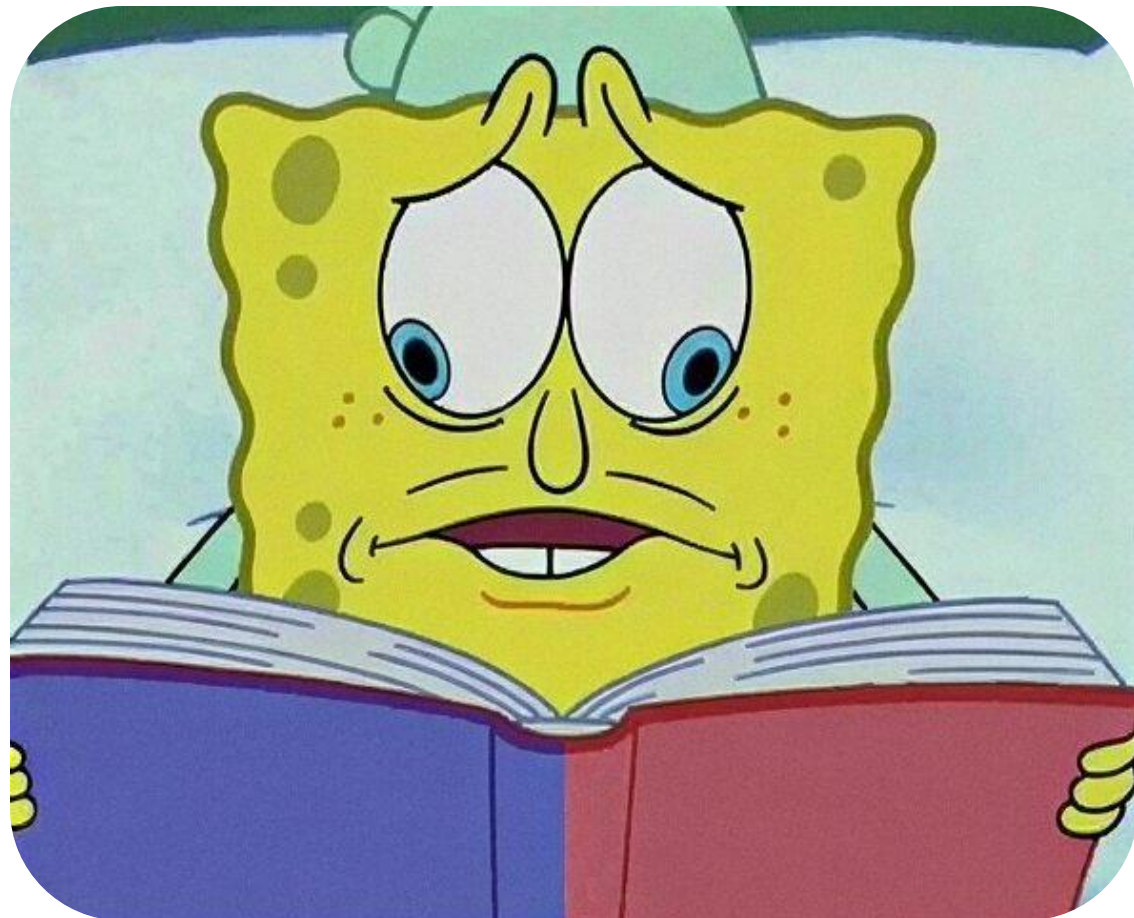
**У одной системы
не может быть
2 главных целей**



Борьба систем убьёт обе метрики

- Система распадётся на 2 подсистемы
- Останется одна, но недостаточная

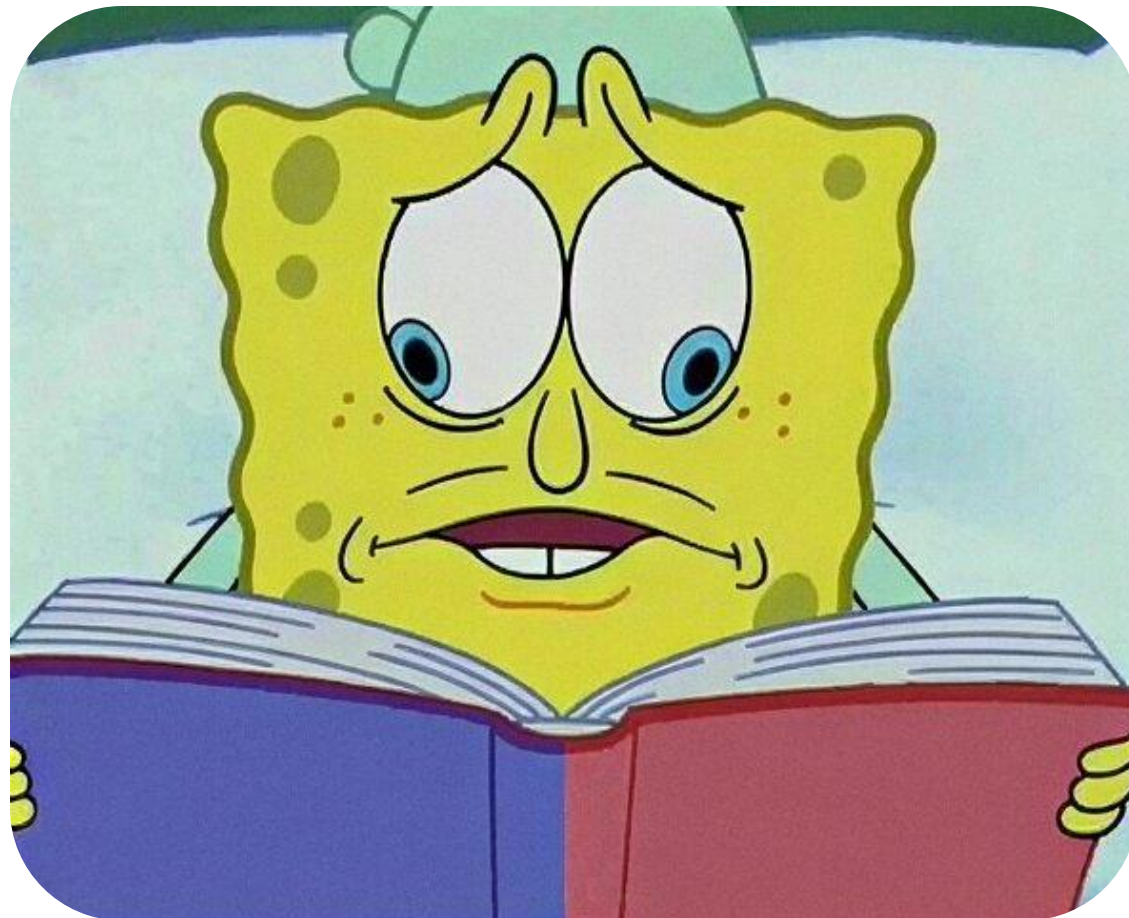
У одной системы не может быть 2 главных целей



Борьба систем убьёт обе метрики

- Система распадётся на 2 подсистемы
- Останется одна, но недостаточная
- Останется одна, но избыточная

У одной системы не может быть 2 главных целей



Борьба систем убьёт обе метрики

- Система распадётся на 2 подсистемы
- Останется одна, но недостаточная
- Останется одна, но избыточная
- Саботаж, партизанская платформа

Сравнивайте строго!

До XX века у слова «приоритет» вообще **не было множественного числа**.

Проект, в котором участвуют несколько сторон, обязательно столкнётся с конфликтом интересов.

История 2.

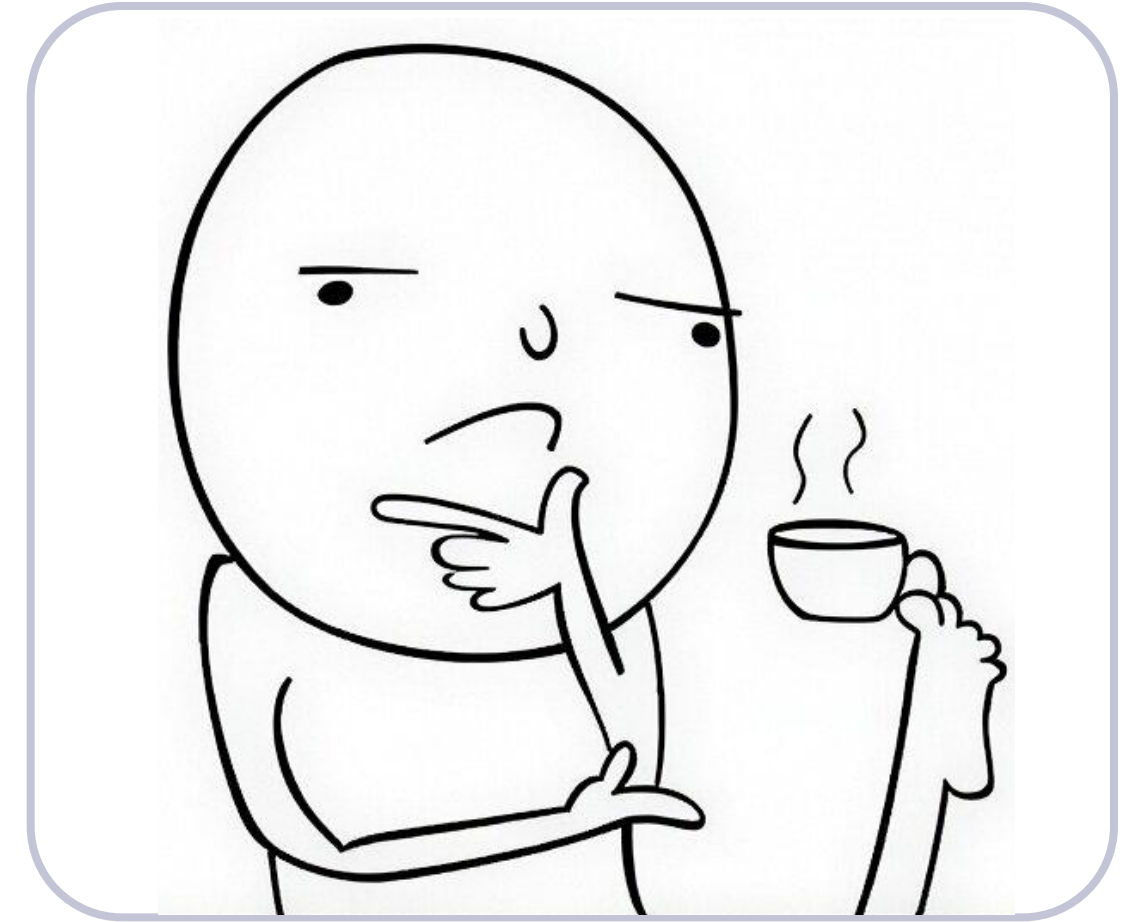
Требования собрать нельзя усреднить



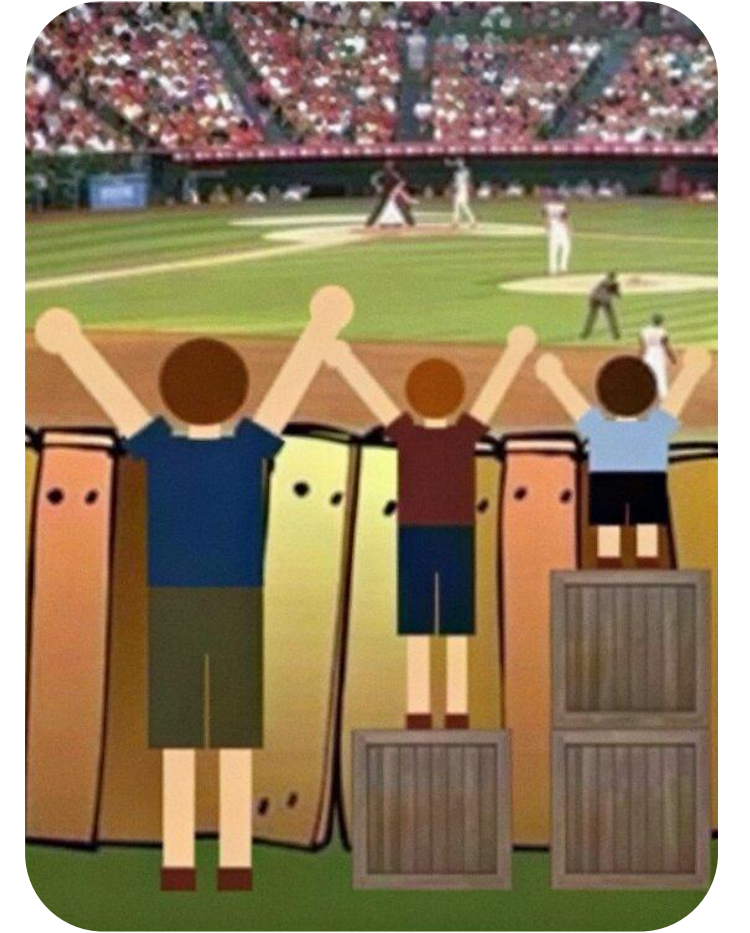
Архитектор подбирает характеристики платформы и компонентов так, чтобы всё покрыть.

Берёт список параметров и смотрит, сколько и чего нужно.

Ну и, усредняя, получает примерные требования.

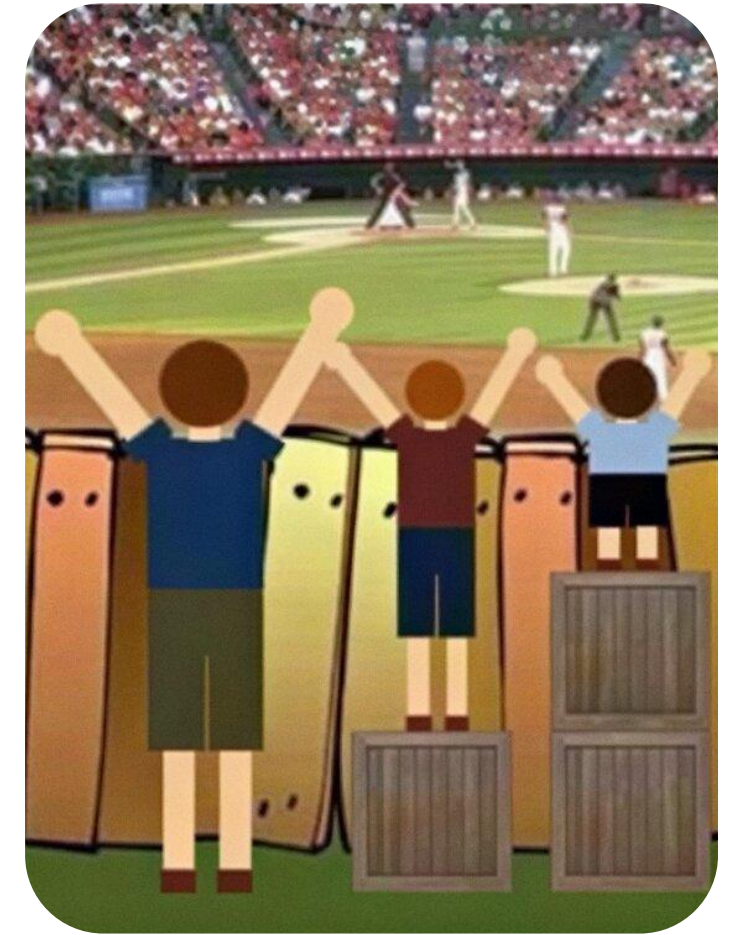


Среднего футболиста
не существует



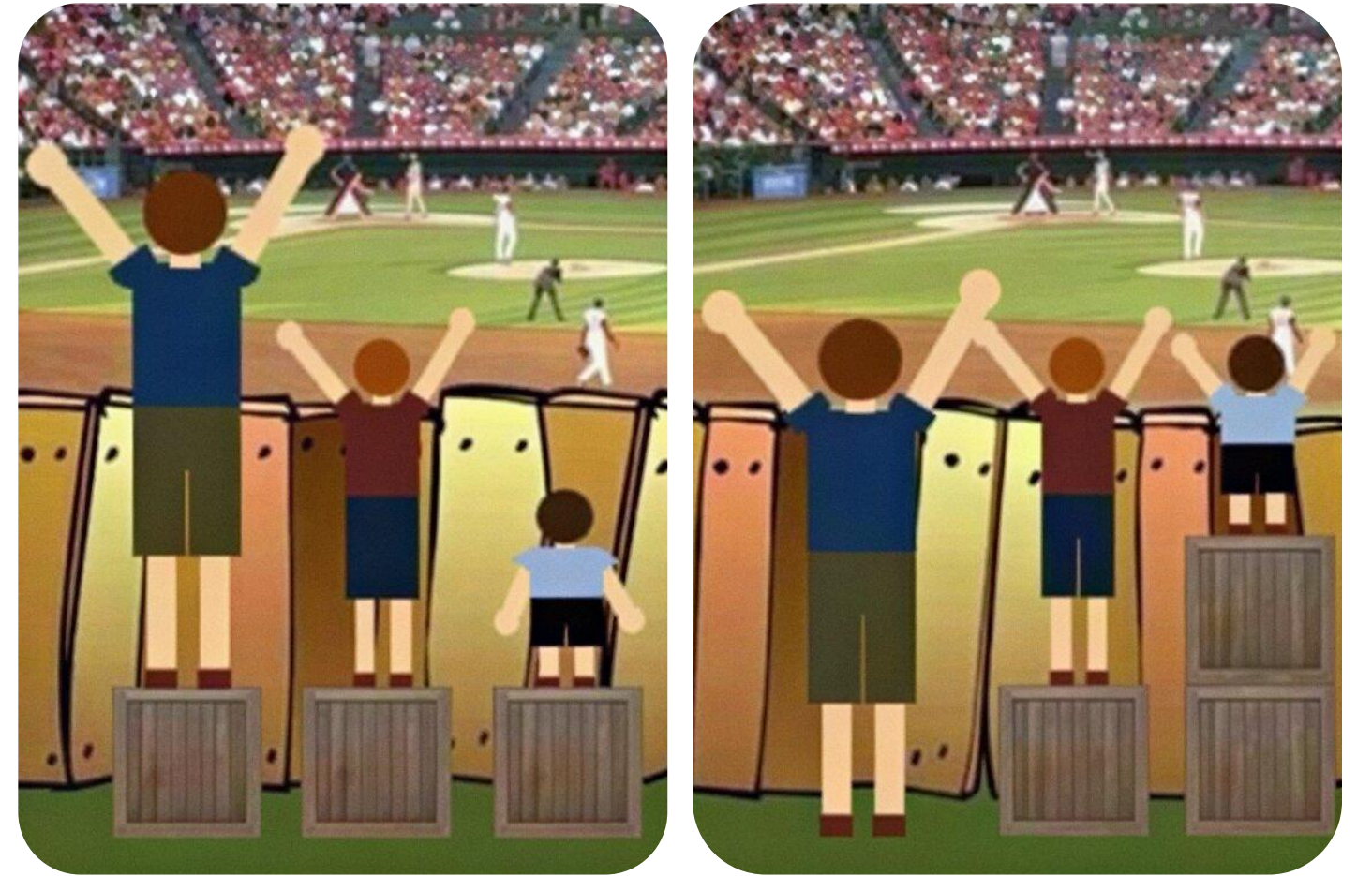
Среднего футболиста не существует

- Для требований важен контекст



Среднего футболиста не существует

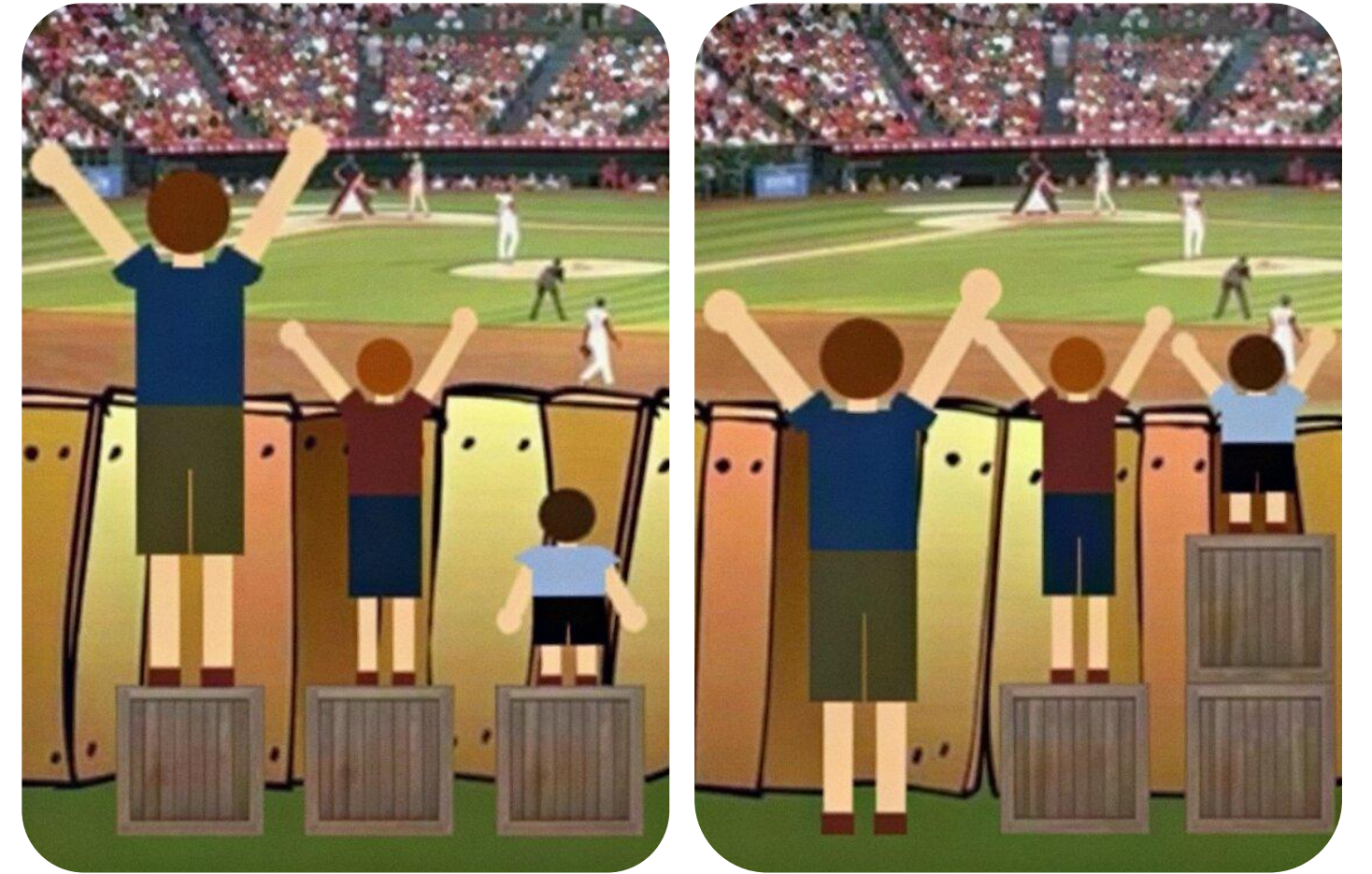
- Для требований важен контекст
- Среднее — это абстракция, которая скрывает контекст



Среднего футболиста не существует

- Для требований важен контекст
- Среднее — это абстракция, которая скрывает контекст
- Кто такой средний пользователь и что ему надо?

Усреднение отбрасывает важные различия!



Цель достигнута,
но не та 😞

Цель достигнута,
но не та 😞

— Опираемся на ложные данные

Цель достигнута,
но не та 😞

- Опираемся на ложные данные
- Получаем несовместимые характеристики

Цель достигнута,
но не та 😞

- Опираемся на ложные данные
- Получаем несовместимые характеристики
- Усреднение портит утилизацию

Определи сильные стороны



Определи сильные стороны

- + Сегментировать по реальному поведению



Определи сильные стороны

- + Сегментировать по реальному поведению
- + Не брать про запас лишнего (поменьше General Purpose)



Определи сильные стороны

- + Сегментировать по реальному поведению
- + Не брать про запас лишнего (поменьше General Purpose)
- + Определить обратную сторону силы

OLAP или OLTP, простое или гибкое



История 3.
Любую проблему можно решить,
добавив ещё один слой абстракции



Есть 2 команды.

У них есть задача — интегрироваться в сервисы друг друга.

Первая настаивает на своём варианте.

Второй он очень не нравится.

Каждый хвалит свой вариант и ругает чужой.

В финале — ни вашим ни нашим. Первая делает свой вариант, а вторая — обёртку для него.

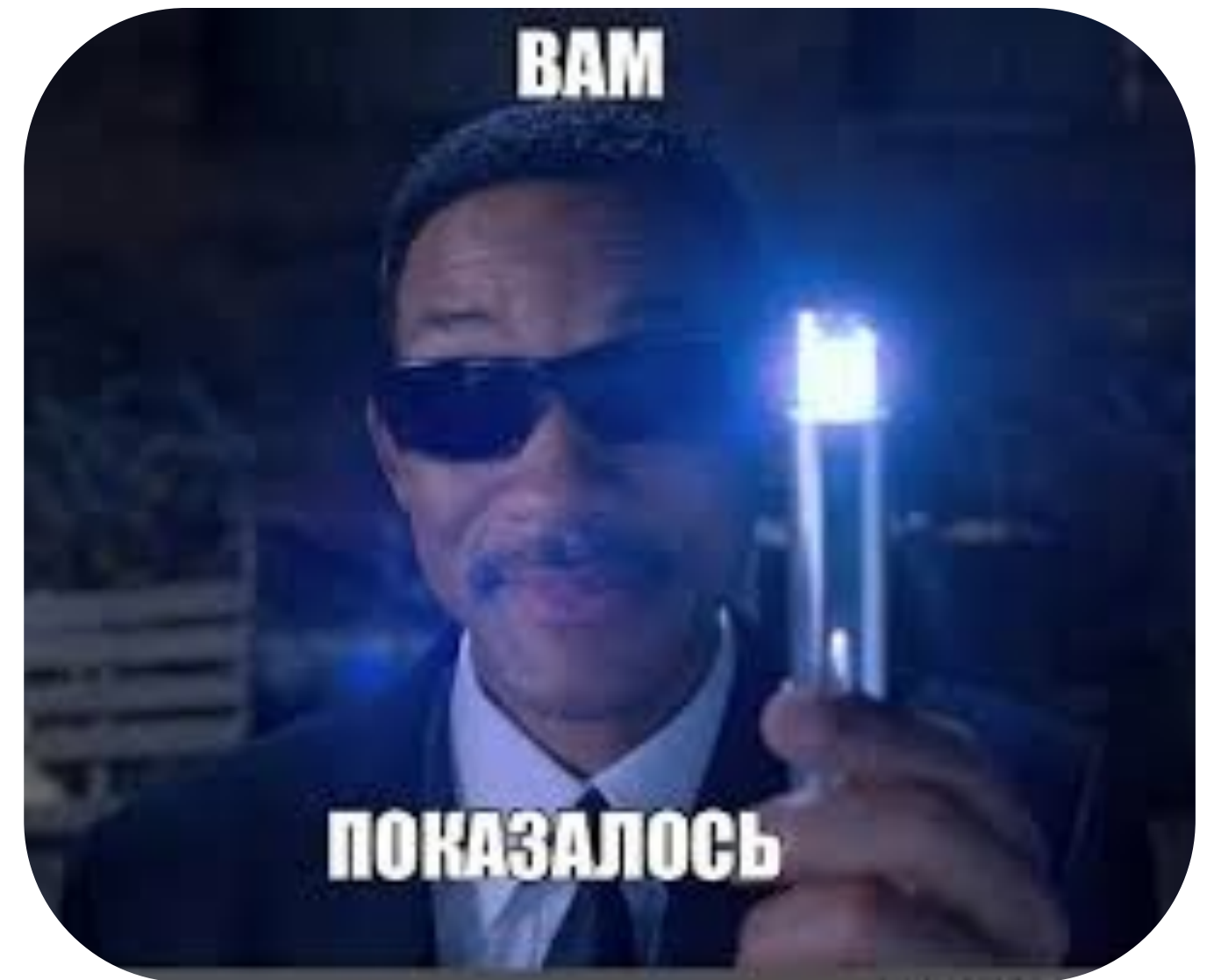


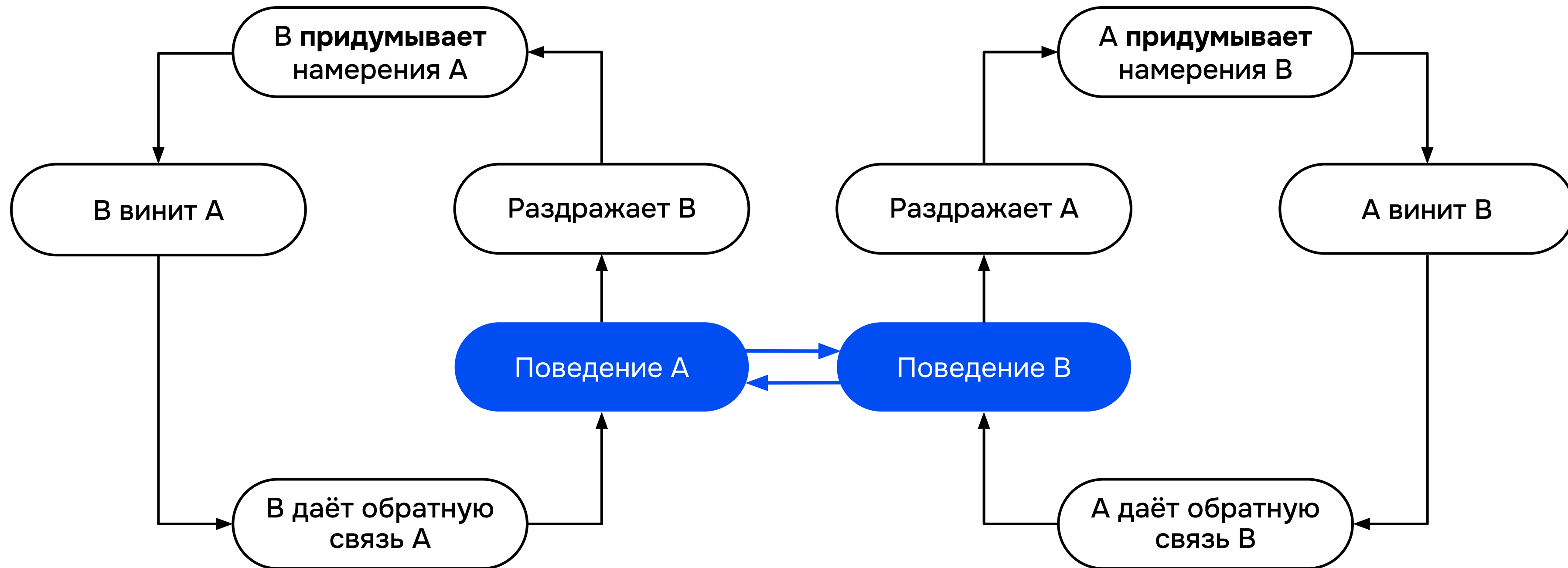
В конфликте всегда есть искажение восприятия

Оцениваем себя по намерениям,
других — по поведению.

Обе команды примерили на себя варианты
и решили, что им хотят испортить систему.

Это всё они!





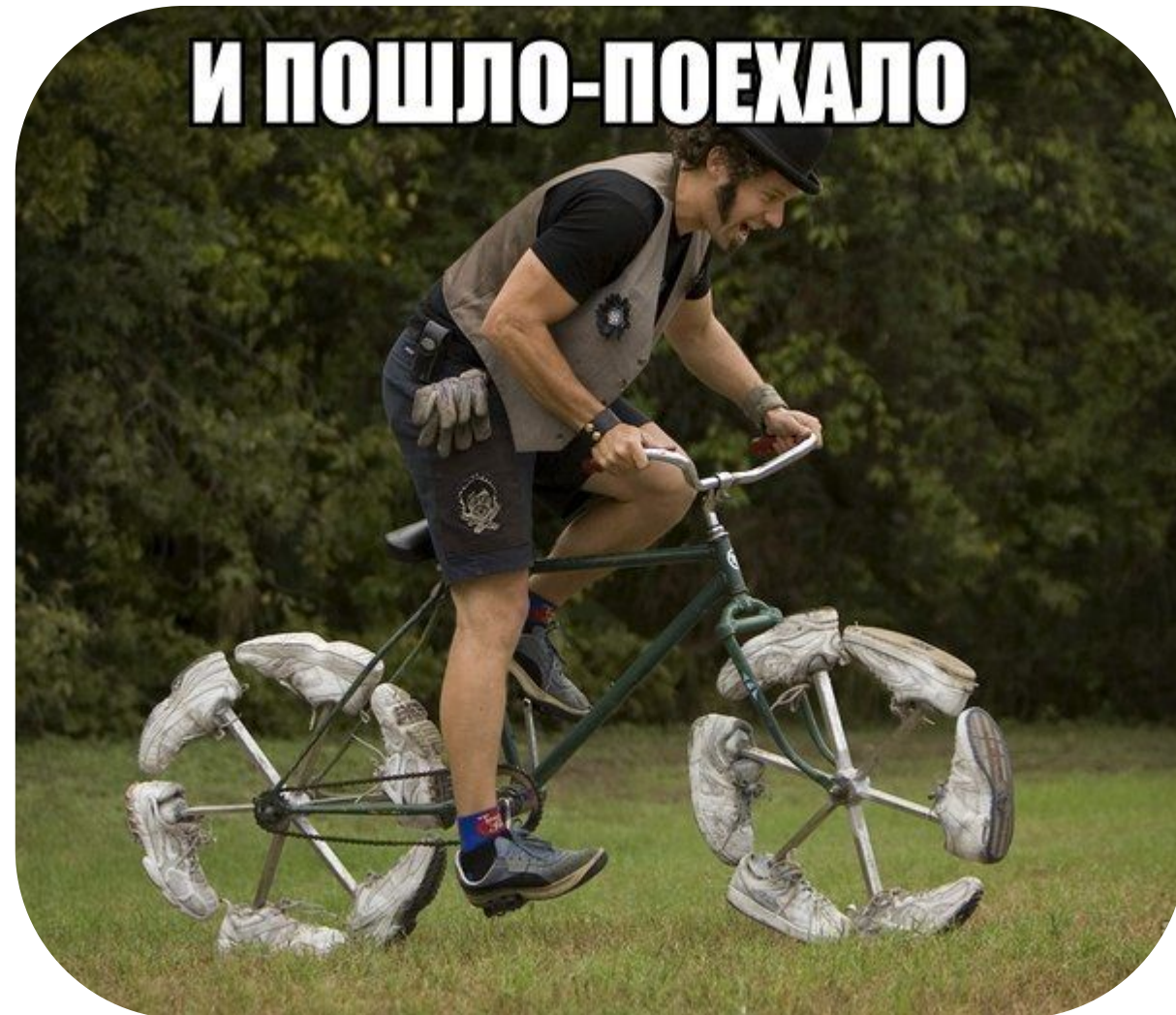
Нерешённый конфликт =
излишнее усложнение
(костыли)

**Нерешённый конфликт
излишнее усложнение
(костыли)** =

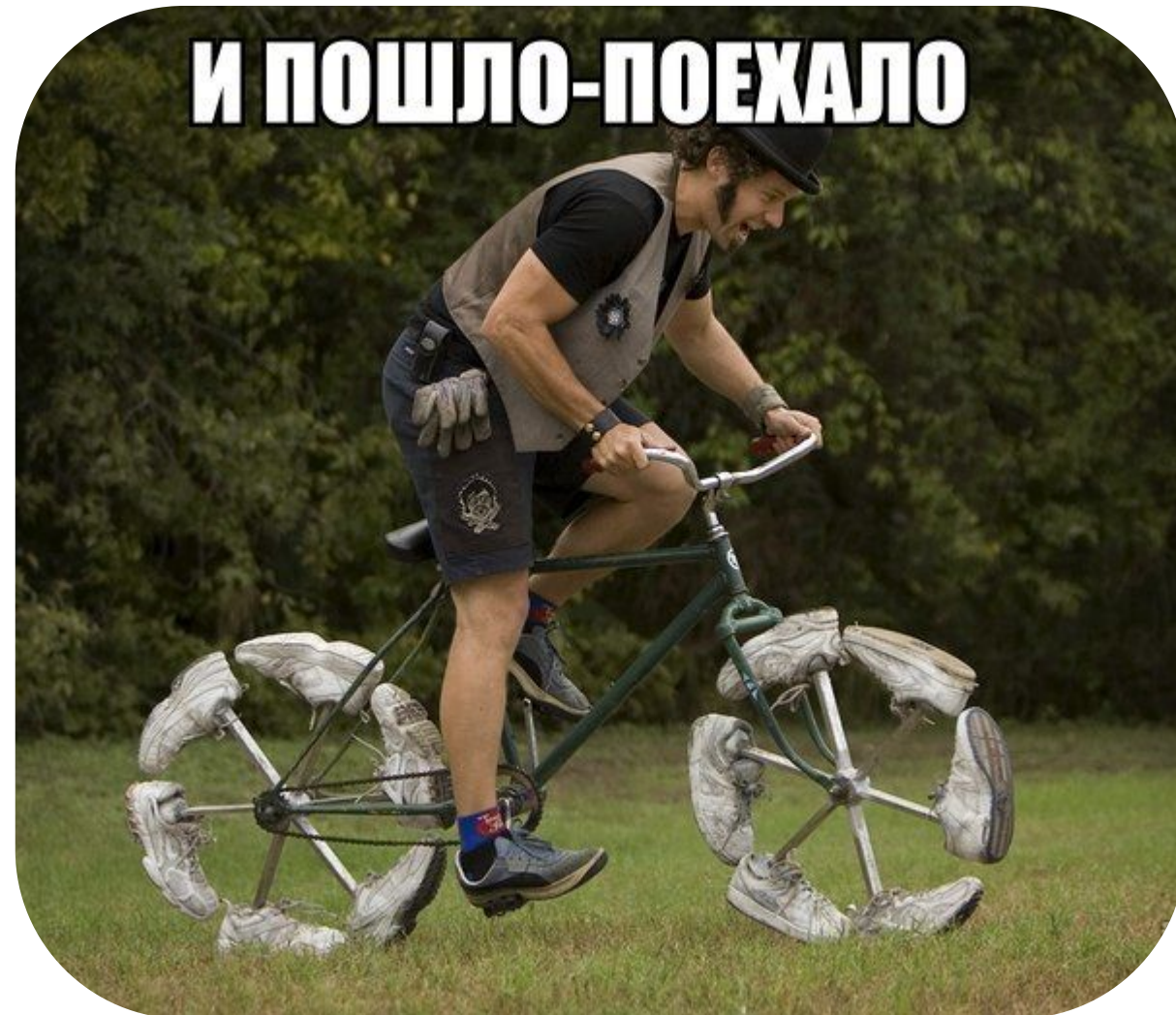
Неясность спецификации говорит о том, что между участниками проекта есть неразрешенные конфликты (Том Демарко).

Нерешённый конфликт излишнее усложнение (костыли) =

Неясность спецификации говорит о том, что между участниками проекта есть неразрешенные конфликты (Том Демарко).



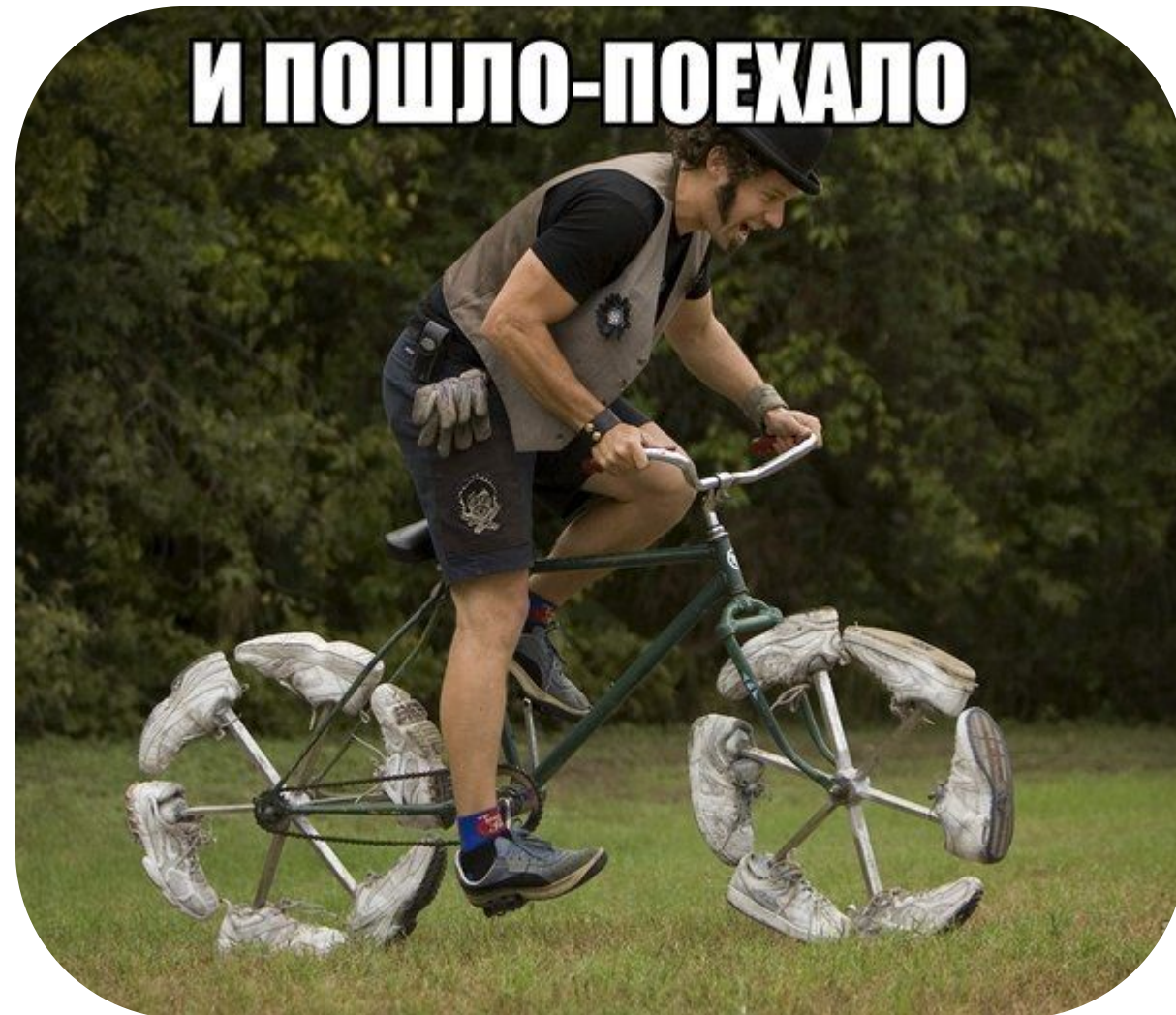
Нерешённый конфликт излишнее усложнение (костыли) =



Неясность спецификации говорит о том, что между участниками проекта есть неразрешенные конфликты (Том Демарко).

- Абстракции над абстракциями (прокси, декораторы, посредники)

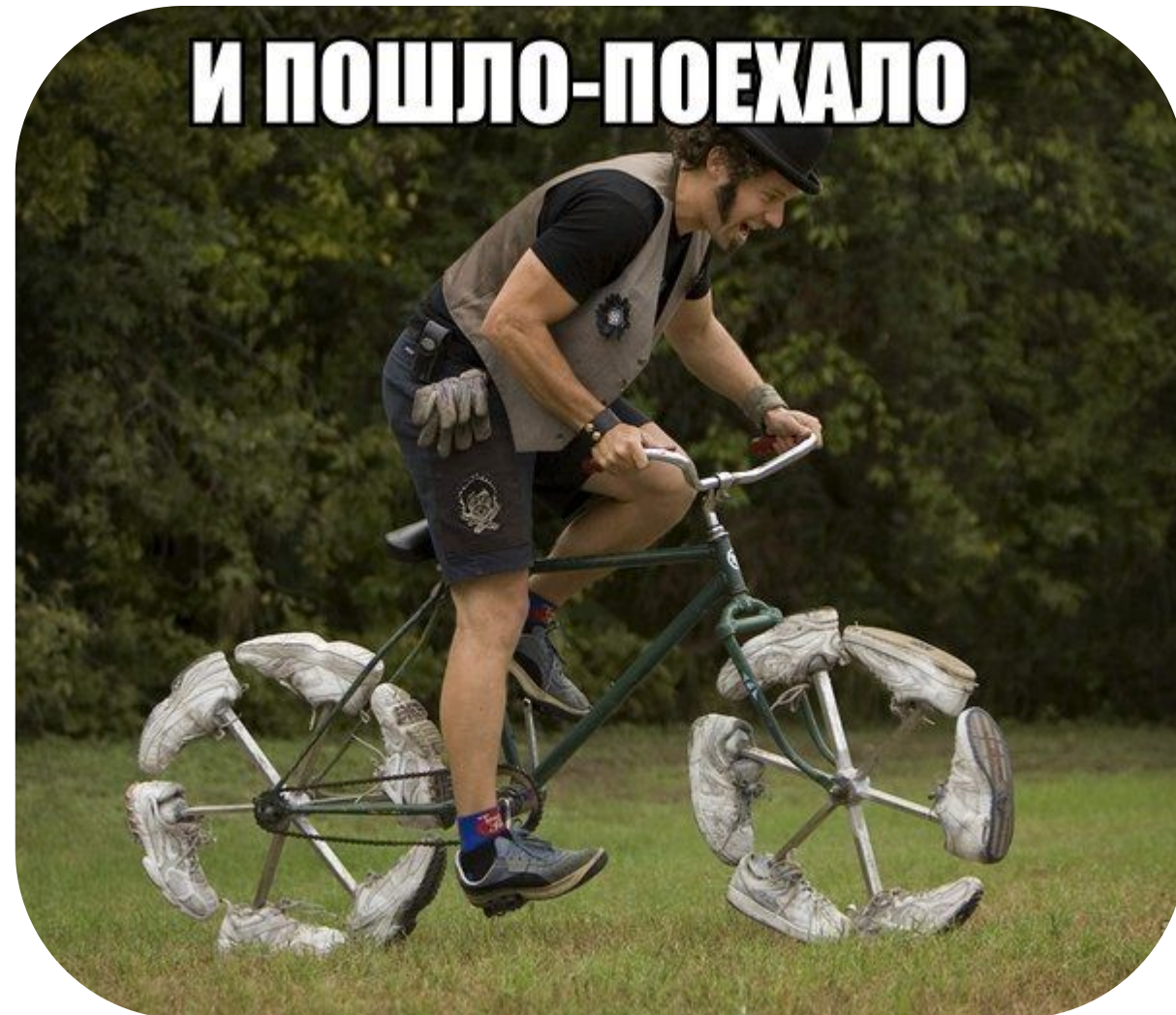
Нерешённый конфликт излишнее усложнение (костыли) =



Неясность спецификации говорит о том, что между участниками проекта есть неразрешенные конфликты (Том Демарко).

- Абстракции над абстракциями (прокси, декораторы, посредники)
- Искажения систем и распространение искажений дальше

Нерешённый конфликт излишнее усложнение (костыли) =



Неясность спецификации говорит о том, что между участниками проекта есть неразрешенные конфликты (Том Демарко).

- Абстракции над абстракциями (прокси, декораторы, посредники)
- Искажения систем и распространение искажений дальше
- Претензии, токсичная атмосфера

Понять и ~~простить~~ решить

Понять и ~~простить~~ решить

✓ Перестать придумывать!

Узнать намерения и логику другой стороны. Нам кажется, что самое страшное — не знать чего-то.

Гораздо хуже быть уверенным, что знаешь, когда это не так

Понять и ~~простить~~ решить

✓ **Перестать придумывать!**

Узнать намерения и логику другой стороны. Нам кажется, что самое страшное — не знать чего-то.

Гораздо хуже быть уверенным, что знаешь, когда это не так

✓ Привлечь авторитетного посредника — единоличного (СТО) или коллективного (коллегия/комитет инженеров)

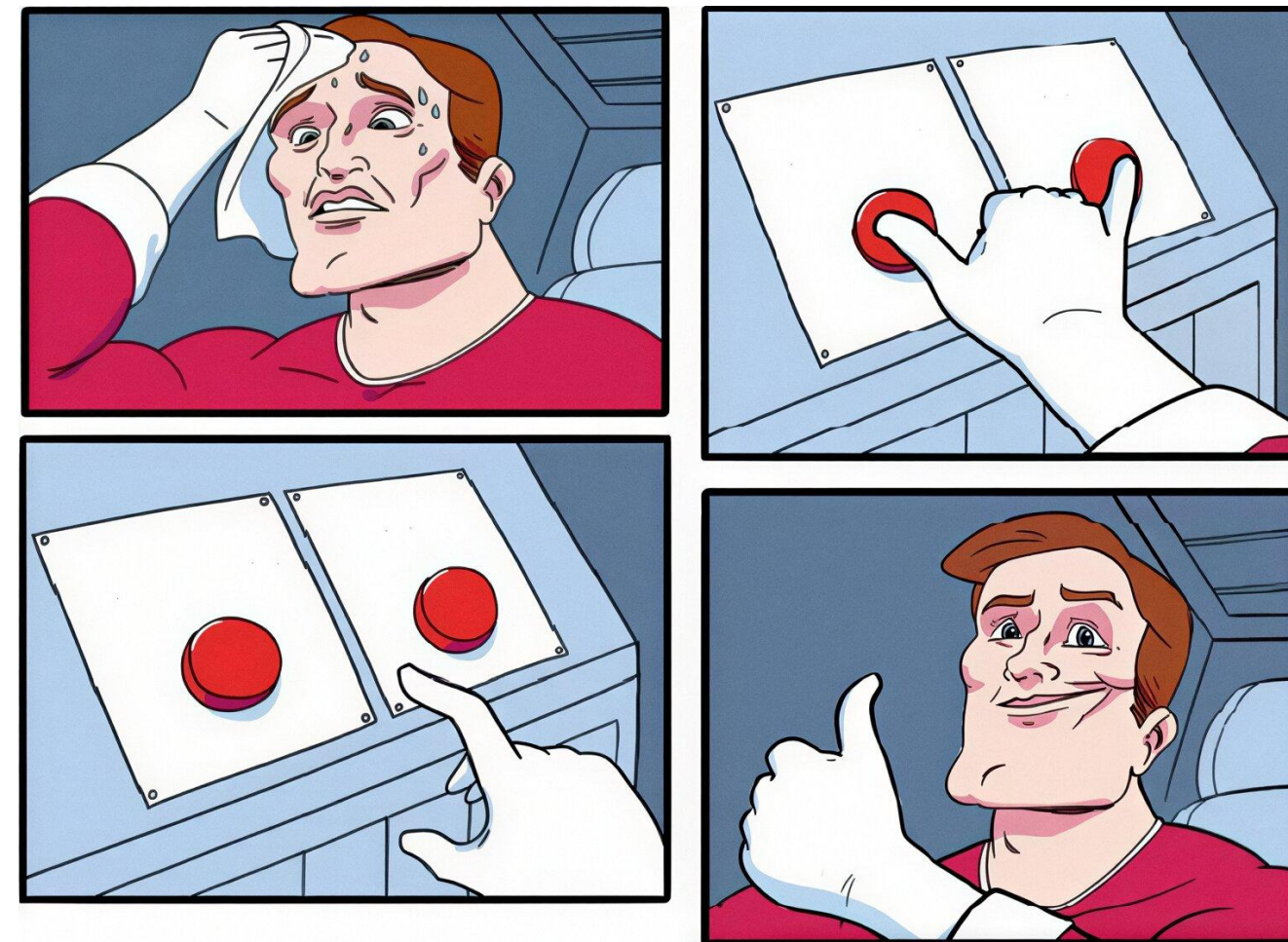
История 4. Нажмите кнопку...



Представьте, что вы сделали кнопку,
по которой можно быстро выполнить какую-
то операцию.

Отдали пользователю. Через некоторое
время смотрите и... «просто приходил
Серёжка, поиграли мы немножко».

Кто-то нажимал кнопку, не отрываясь.



Трагедия систем коллективного использования



Трагедия систем коллективного использования



- Каждый действует наилучшим образом в собственных интересах

Трагедия систем коллективного использования



- Каждый действует наилучшим образом в собственных интересах
- Достигается пороговый уровень спроса, в системе не остаётся запаса ресурсов

Трагедия систем коллективного использования



- Каждый действует наилучшим образом в собственных интересах
- Достигается пороговый уровень спроса, в системе не остаётся запаса ресурсов
- Каждое новое использование уменьшает выгоду всех остальных

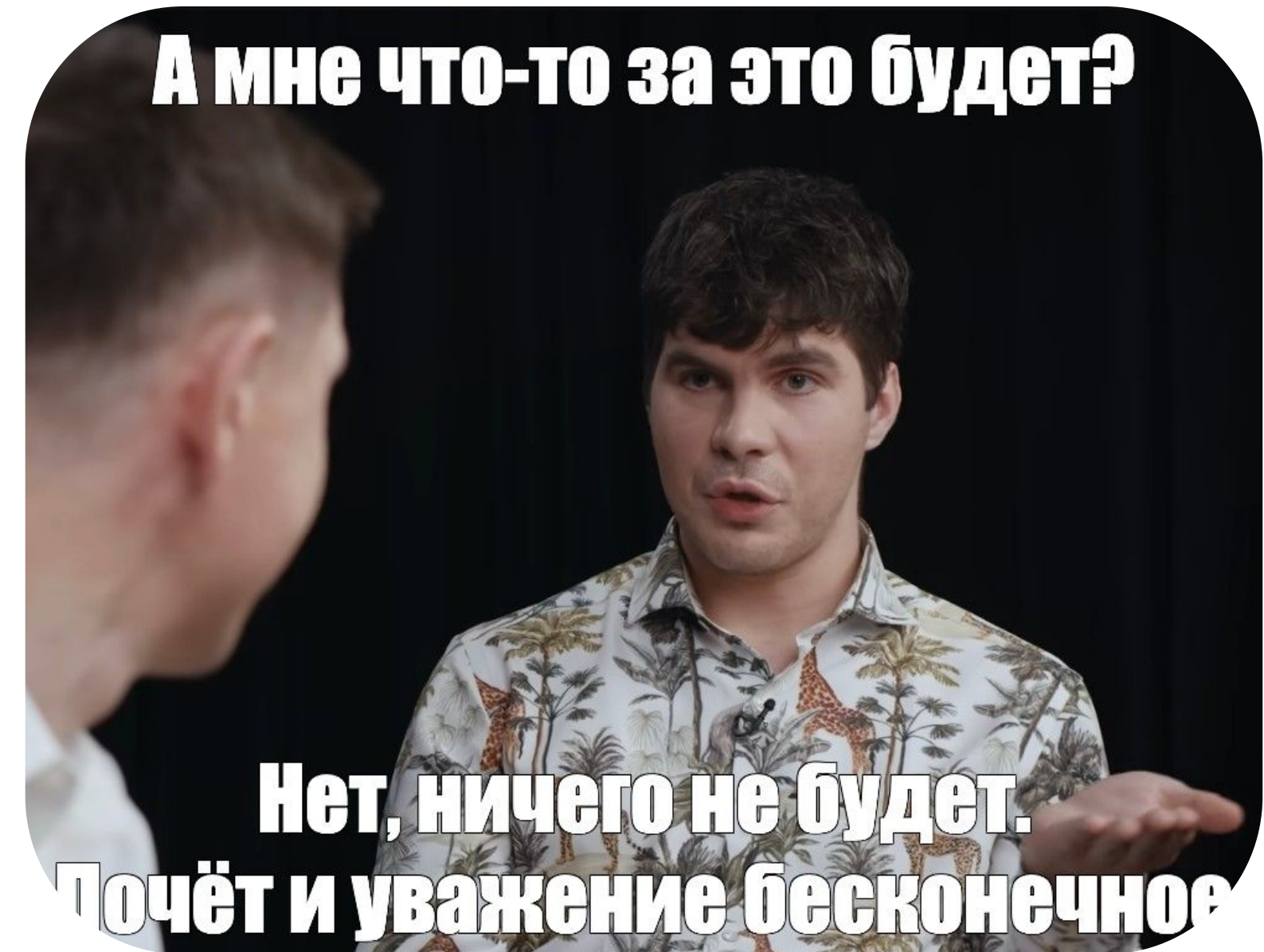
Хайлоад на ровном месте

Хайлоад на ровном месте

Убираем препятствия. Цена действия
снижается, спрос – растёт.

Хайлоад на ровном месте

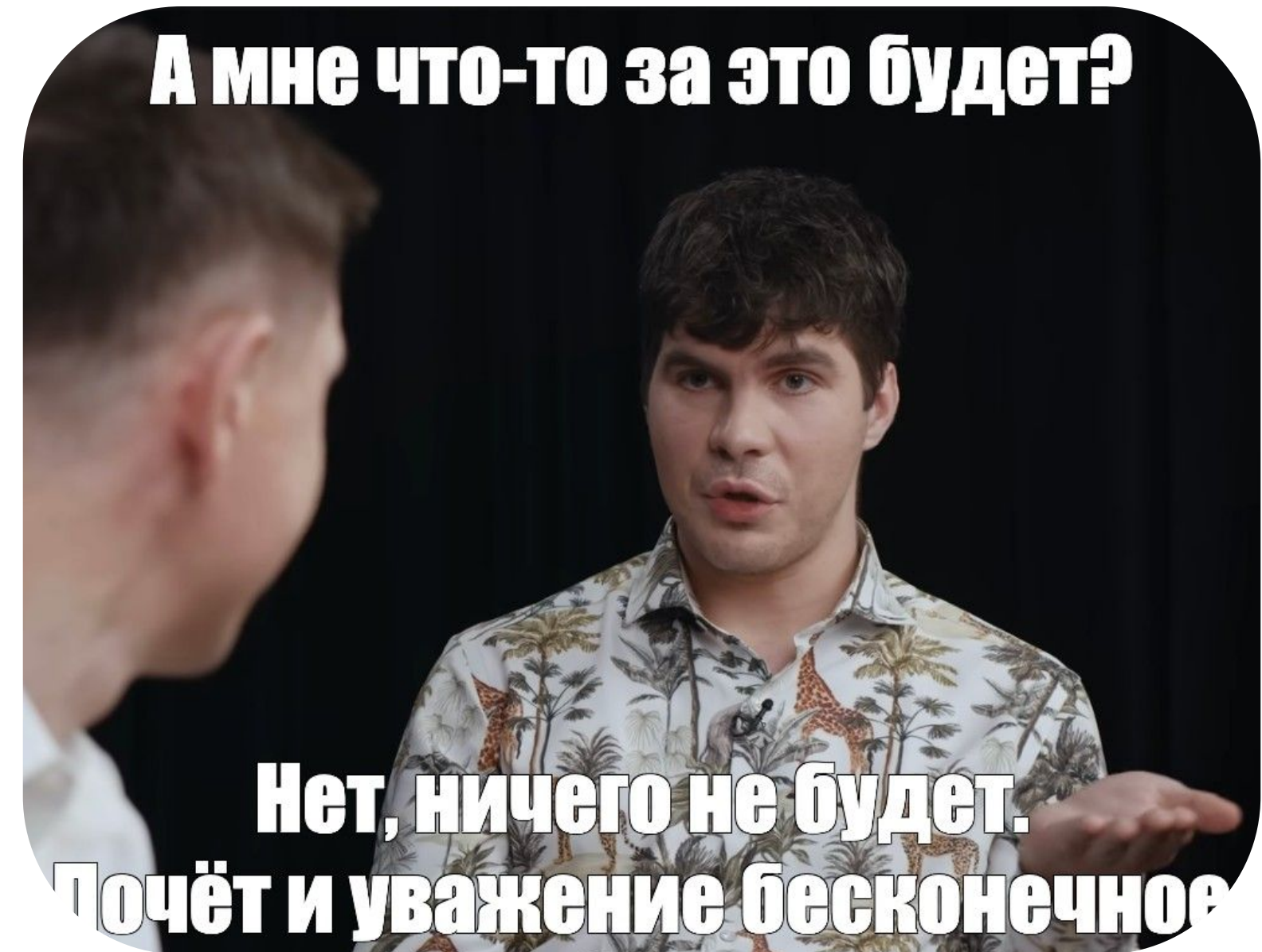
Убираем препятствия. Цена действия снижается, спрос – растёт.



Хайлоад на ровном месте

Убираем препятствия. Цена действия снижается, спрос – растёт.

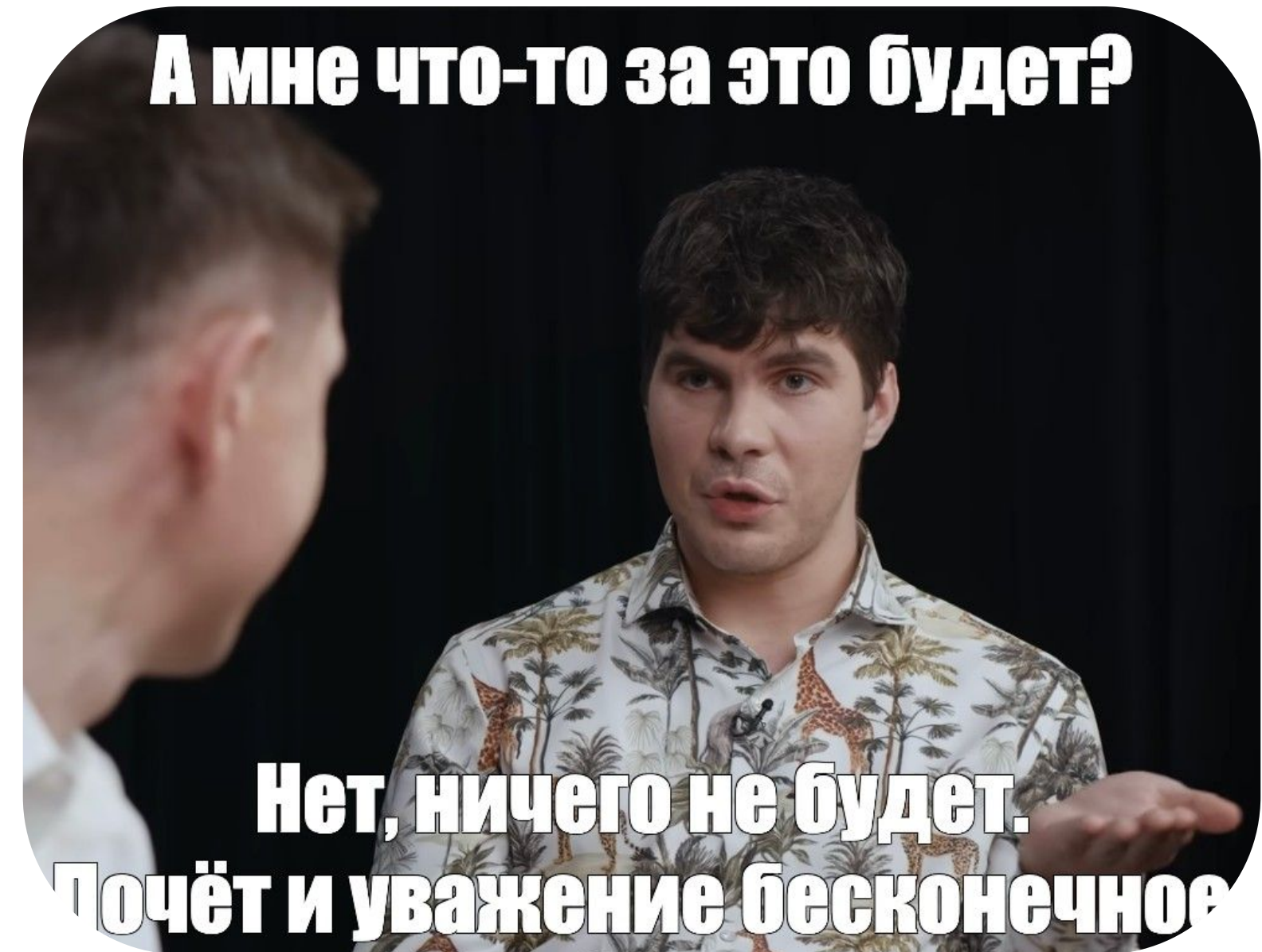
- Завести шаренные ресурсы и ждать, когда кто-нибудь проявит себя и положит соседей



Хайлоад на ровном месте

Убираем препятствия. Цена действия снижается, спрос – растёт.

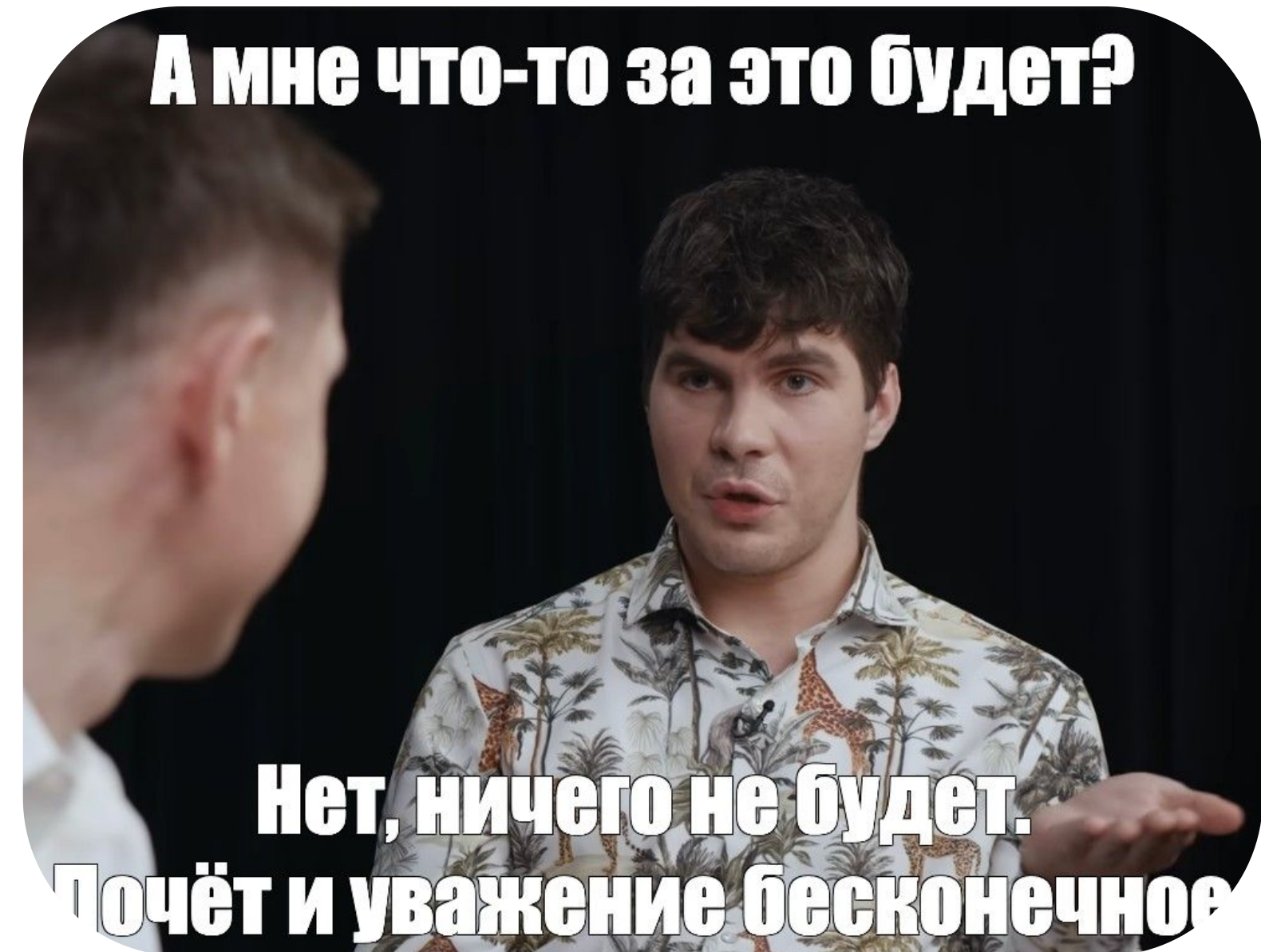
- Завести шаренные ресурсы и ждать, когда кто-нибудь проявит себя и положит соседей
- Общая система для конфигов



Хайлоад на ровном месте

Убираем препятствия. Цена действия снижается, спрос – растёт.

- Завести шаренные ресурсы и ждать, когда кто-нибудь проявит себя и положит соседей
- Общая система для конфигов
- Шаблон для сервисов



Регулировать спрос

Регулировать спрос

Не убивая самообслуживание!



Регулировать спрос

Не убивая самообслуживание!

- ✓ Не перекладывать контроль на пользователей



Регулировать спрос

Не убивая самообслуживание!

- ✓ Не перекладывать контроль на пользователей
- ✓ Бюджетировать (ставить лимиты)



Регулировать спрос

Не убивая самообслуживание!

- ✓ Не перекладывать контроль на пользователей
- ✓ Бюджетировать (ставить лимиты)
- ✓ Ставить перегородки



Регулировать спрос

Не убивая самообслуживание!

- ✓ Не перекладывать контроль на пользователей
- ✓ Бюджетировать (ставить лимиты)
- ✓ Ставить перегородки
- ✓ Ограничивать полномочия

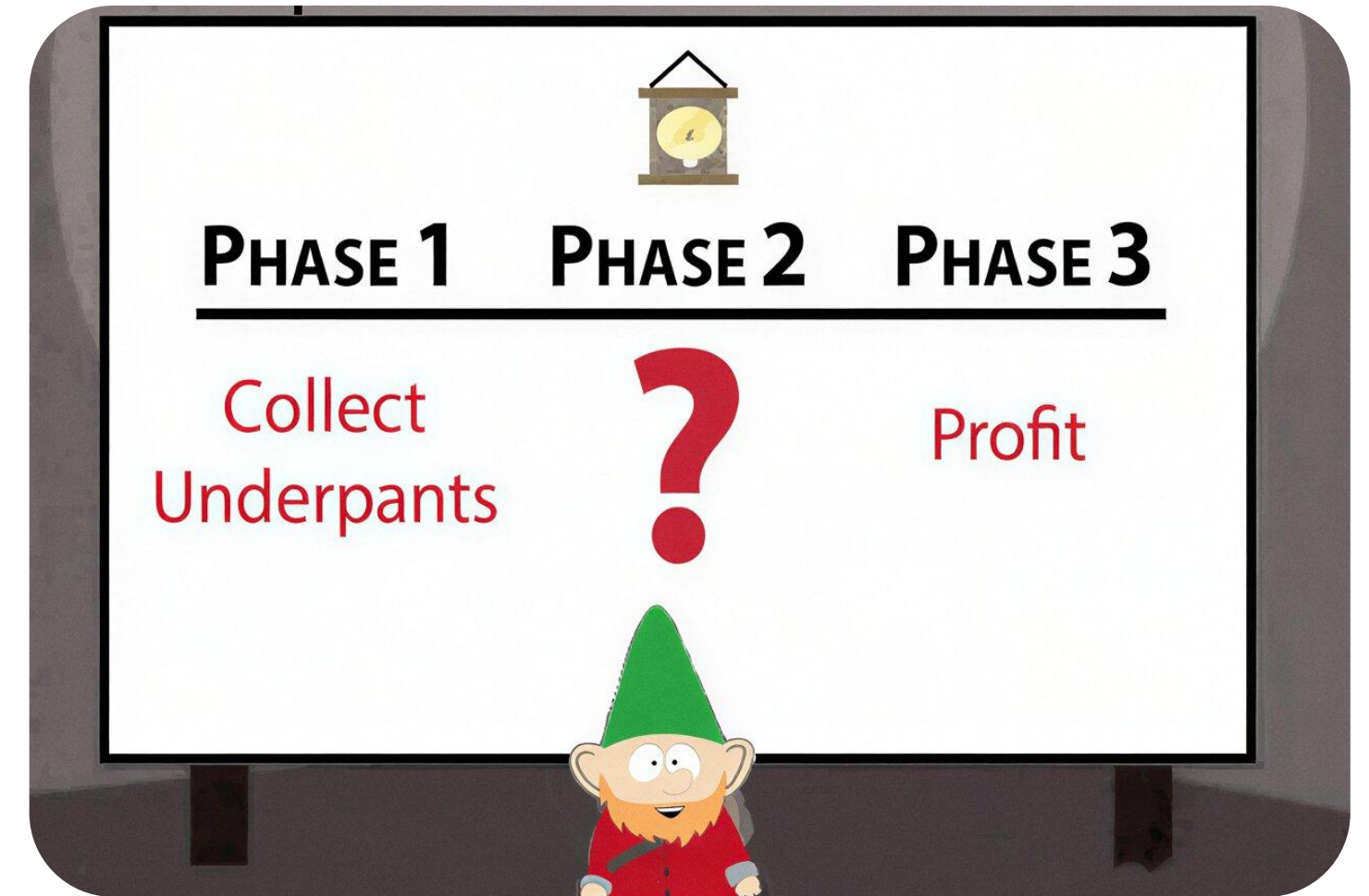


История 5. А что потом?



А что потом?

1. Собираем платформу
2. ...
3. Profit!



Day 0



Day 0



Day 1



Day 0



Day 1



Day 2



Проблема не в создании, а в поддержании порядка



**Мы слишком сильно
упрощаем то,
что нам нравится**

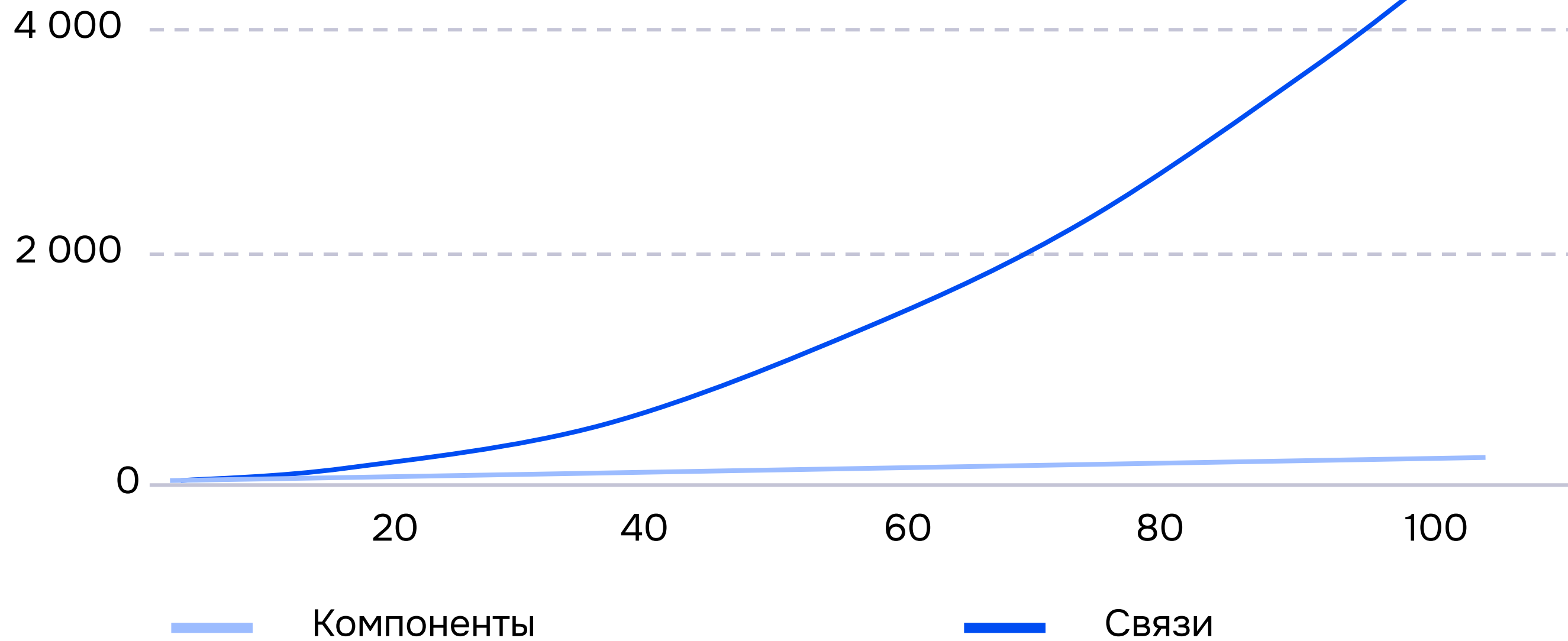


Создавать платформы — это интересно!
Тут много творчества и энтузиазма.

А внедрять и поддерживать? Не очень.
Тут дисциплина и порядок.

x2 платформа != x2 ресурсы

Динамическая сложность растёт быстрее

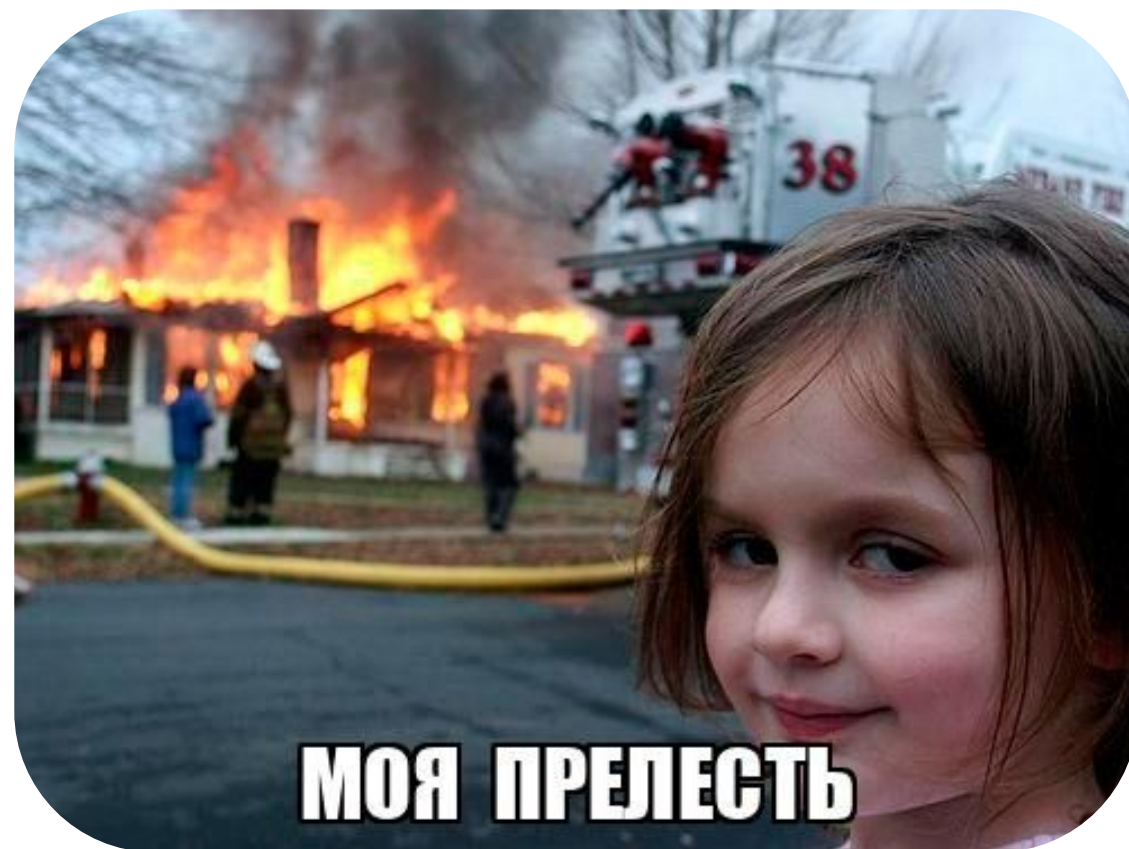


**Стоимость владения
может быть намного
больше стоимости
создания**

**Стоимость владения
может быть намного
больше стоимости
создания**

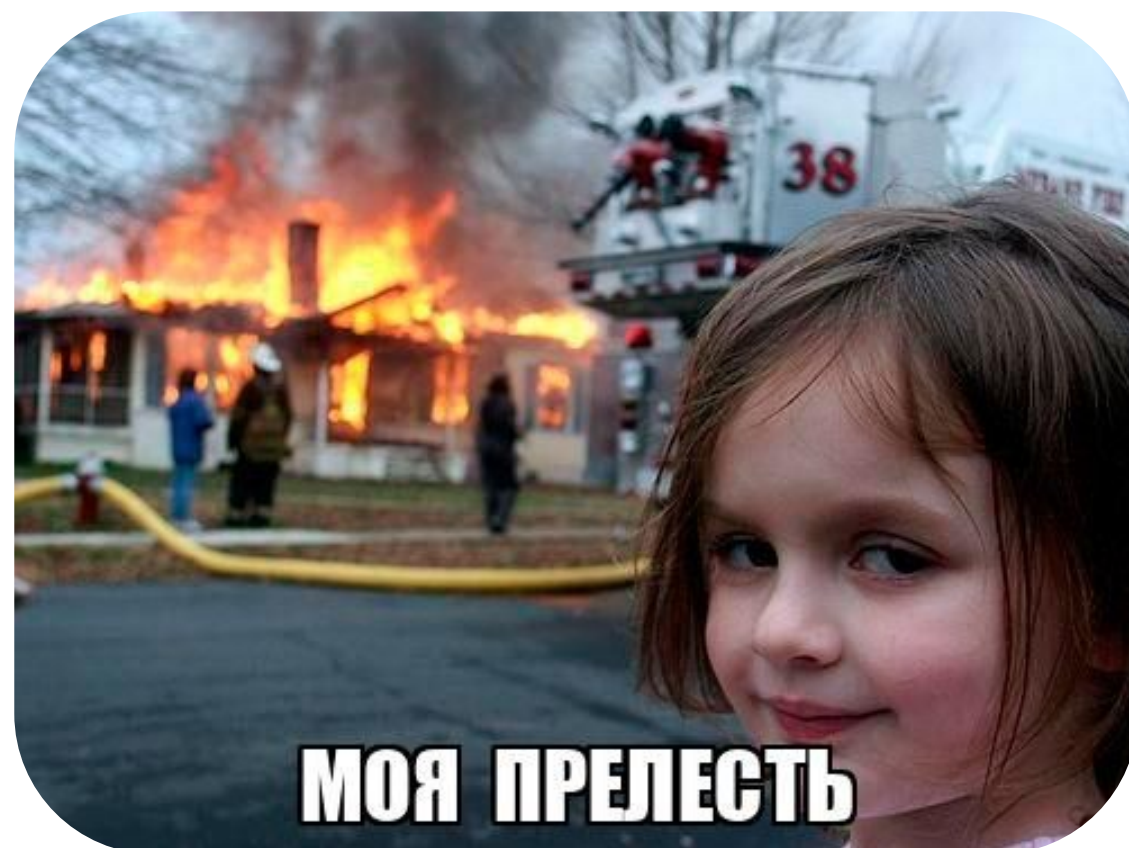
- Свободное время и энтузиазм внезапно могут закончиться

Стоимость владения может быть намного больше стоимости создания



- Свободное время и энтузиазм внезапно могут закончиться
- Перегруз приведёт к выгоранию

Стоимость владения может быть намного больше стоимости создания



- Свободное время и энтузиазм внезапно могут закончиться
- Перегруз приведёт к выгоранию
- И бросить жалко и продолжать тяжело

Планировать по силам, а не по обстоятельствам

Чем раньше прекратить ненужную работу, тем лучше

Планировать по силам, а не по обстоятельствам

Чем раньше прекратить ненужную работу, тем лучше

- ✓ Оценить всё, что можно оценить в вашем случае

Планировать по силам, а не по обстоятельствам

Чем раньше прекратить ненужную работу, тем лучше

✓ Оценить всё, что можно оценить в вашем случае

✓ Выбросить лишнее, оставить главное

Планировать по силам, а не по обстоятельствам

Чем раньше прекратить ненужную работу, тем лучше

✓ Оценить всё, что можно оценить в вашем случае

✓ Выбросить лишнее, оставить главное

✓ Переложить поддержку

Day-2. Production ready – 80% усилий

Мы сделали на этом продукт

Deckhouse Kubernetes Platform

Community Edition

Kubernetes – это же 5 бинарников, а потом...

хранилище

сеть

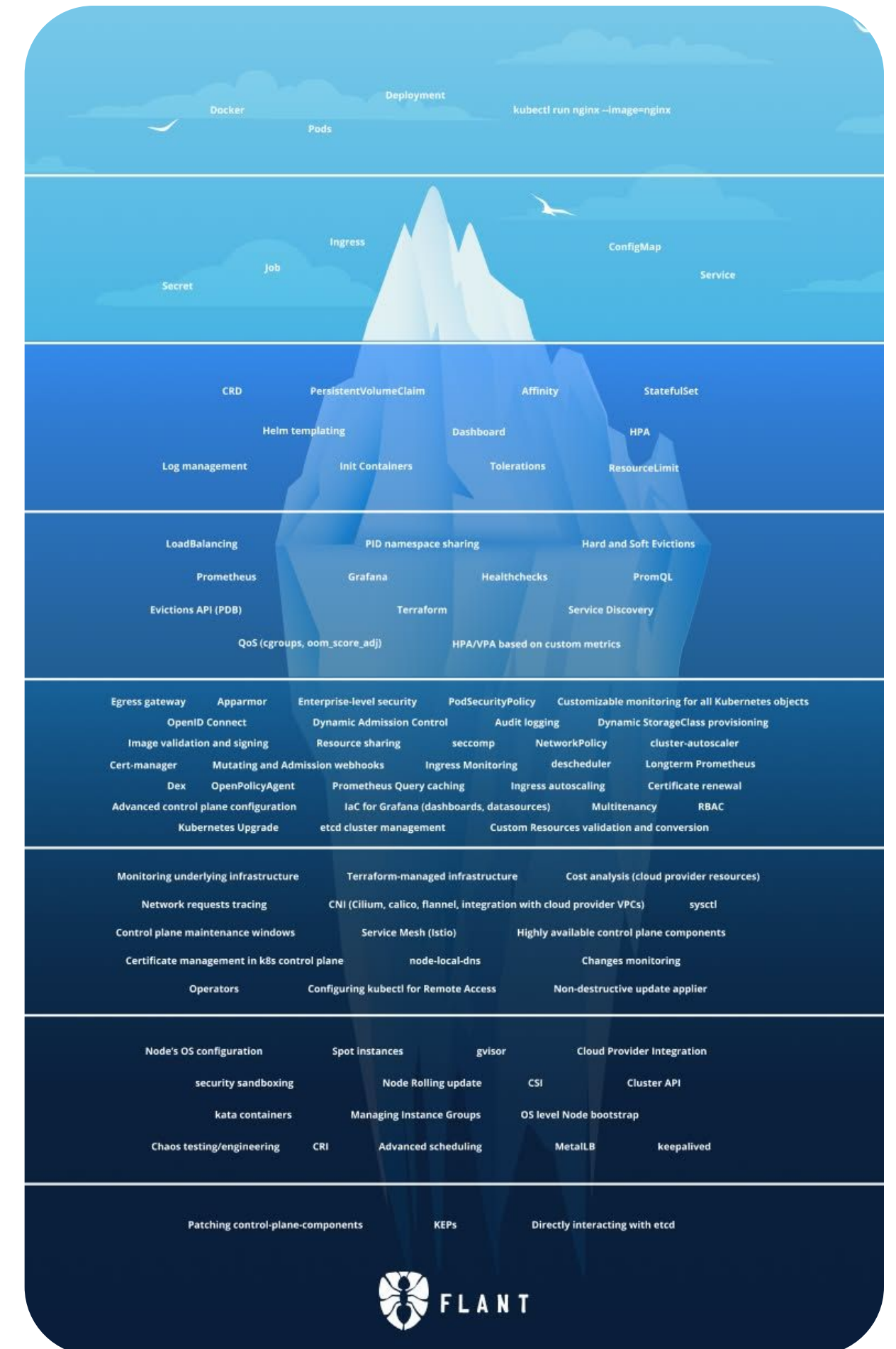
наблюдаемость

безопасность

виртуализация

облака

etc.



Чтобы делать хорошо, нужны ресурсы и система

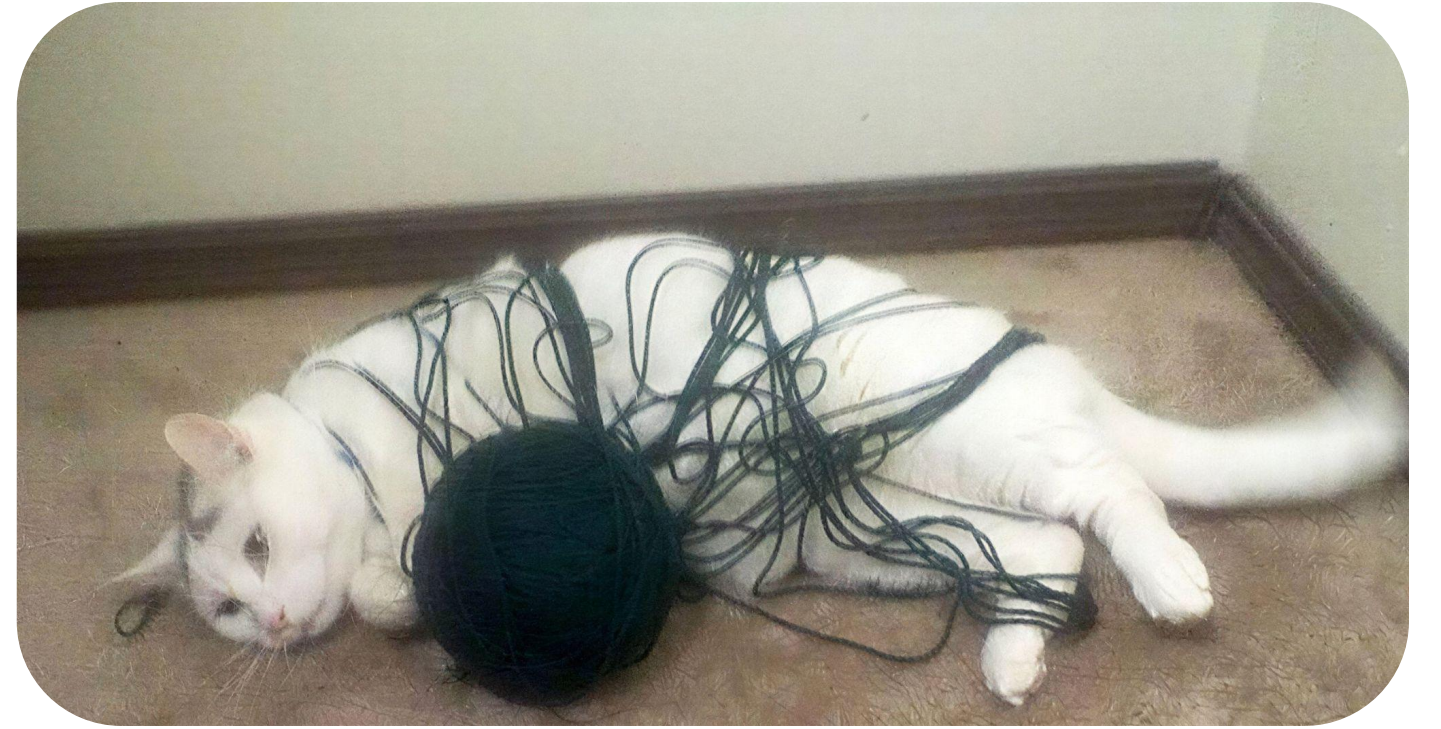
ОС x Редакция DKP x
x Версия K8s x Облачный провайдер =
= 100 TestSuite'ов только
на деплой в облака

- ✓ 150 человек
- ✓ Автотесты (покрытие 70–95 %, дневной и ночной набор)
- ✓ Тестирование совместимости
- ✓ Релизные каналы (Alpha, Beta, EA, Stable, Rock Solid)

История 6. Груз ответственности



Время монолитов прошло.
И сейчас мы пилим множество сервисов.
Но... Почему-то...
Изменять эти сервисы всё равно тяжело 😞



Стабильность
добровольная
и принудительная

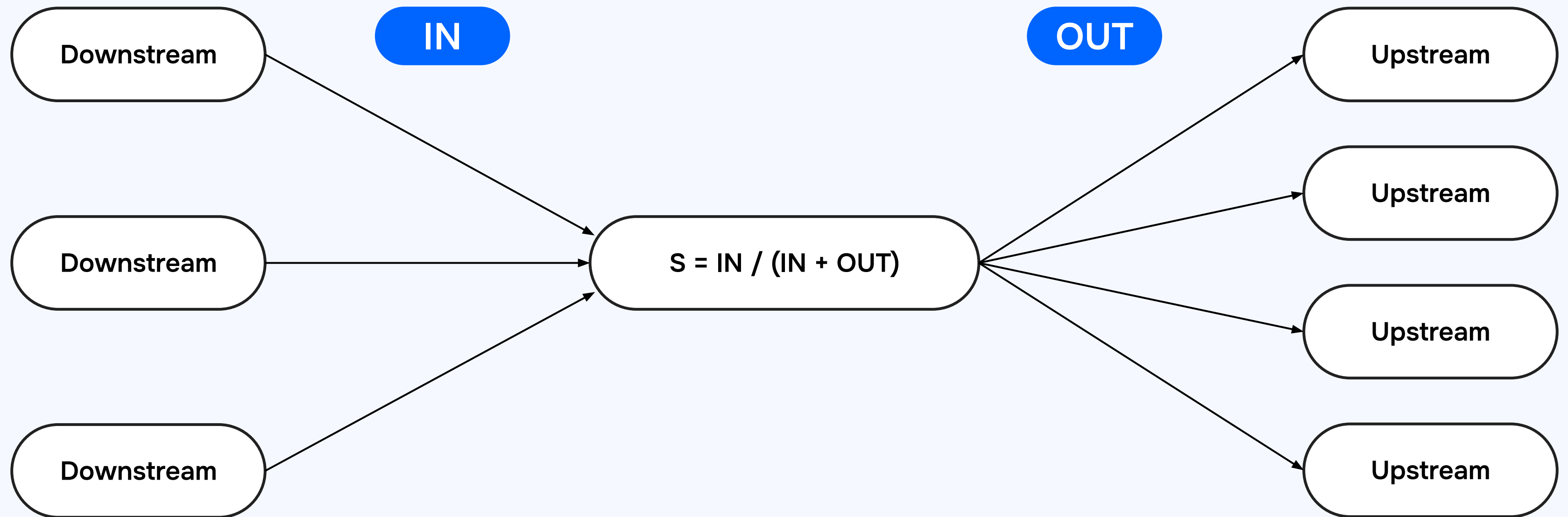
Стабильность добровольная и принудительная

- ✓ Стабильность как **надёжность**.
Компонент, который хорошо протестирован, в работоспособности которого можно не сомневаться

Стабильность добровольная и принудительная

- ✓ Стабильность как **надёжность**.
Компонент, который хорошо протестирован, в работоспособности которого можно не сомневаться
- ✓ Стабильность как **инерция**. Следствие зависимостей. Поменять у дома крышу сильно легче, чем фундамент

Считаем стабильность



**Больно, когда эти две стабильности
не сходятся в одном компоненте**

Больно, когда эти две стабильности не сходятся в одном компоненте

- Компонент постоянно сыпется

Больно, когда эти две стабильности не сходятся в одном компоненте

- Компонент постоянно сыпется

- Менять его очень сложно

Больно, когда эти две стабильности не сходятся в одном компоненте

- Компонент постоянно сыпется

- Менять его очень сложно

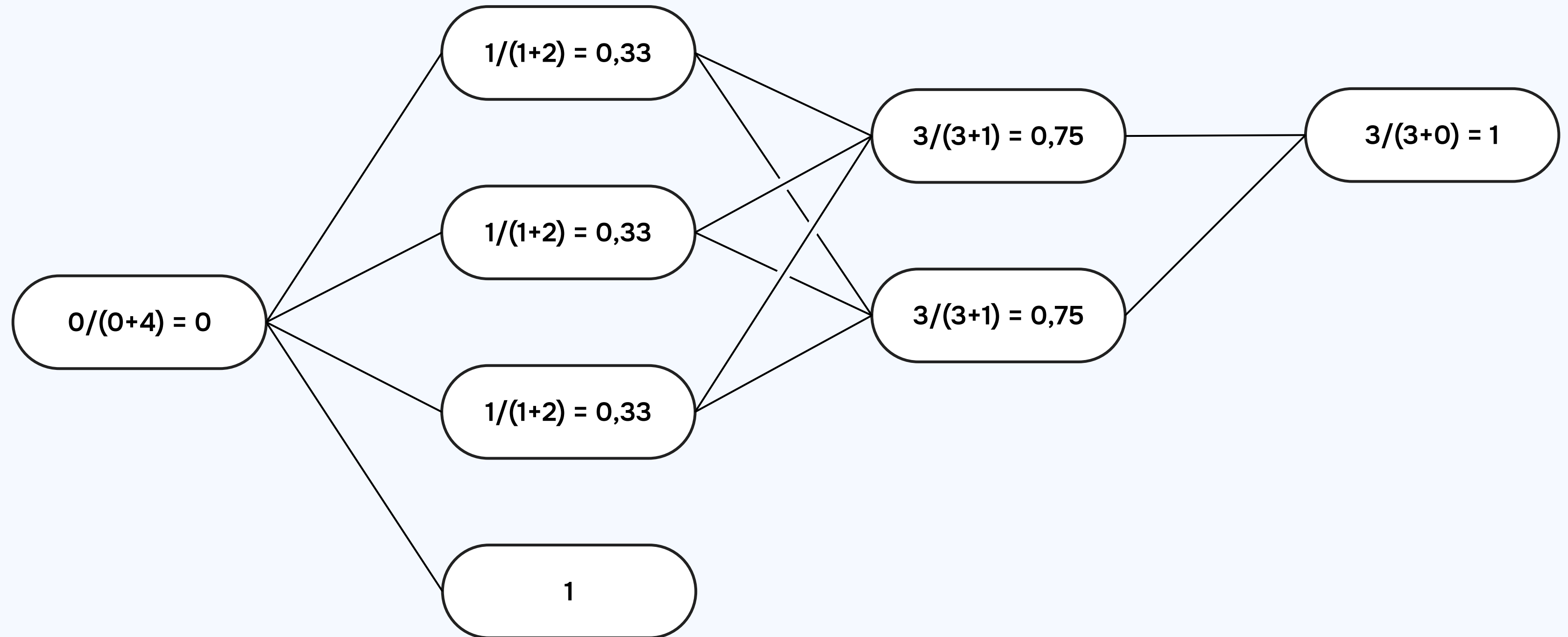
- Увлекательный дебаг

Представление/
агрегация

Управление/
процесс

Сервис/
сущность

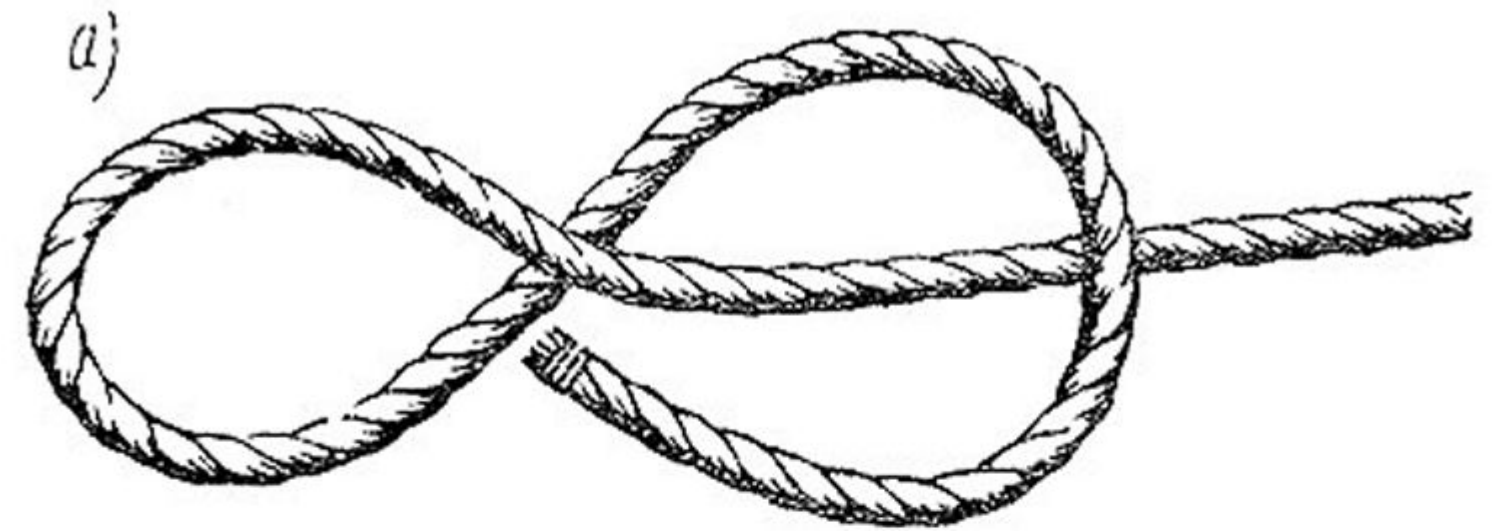
Функция

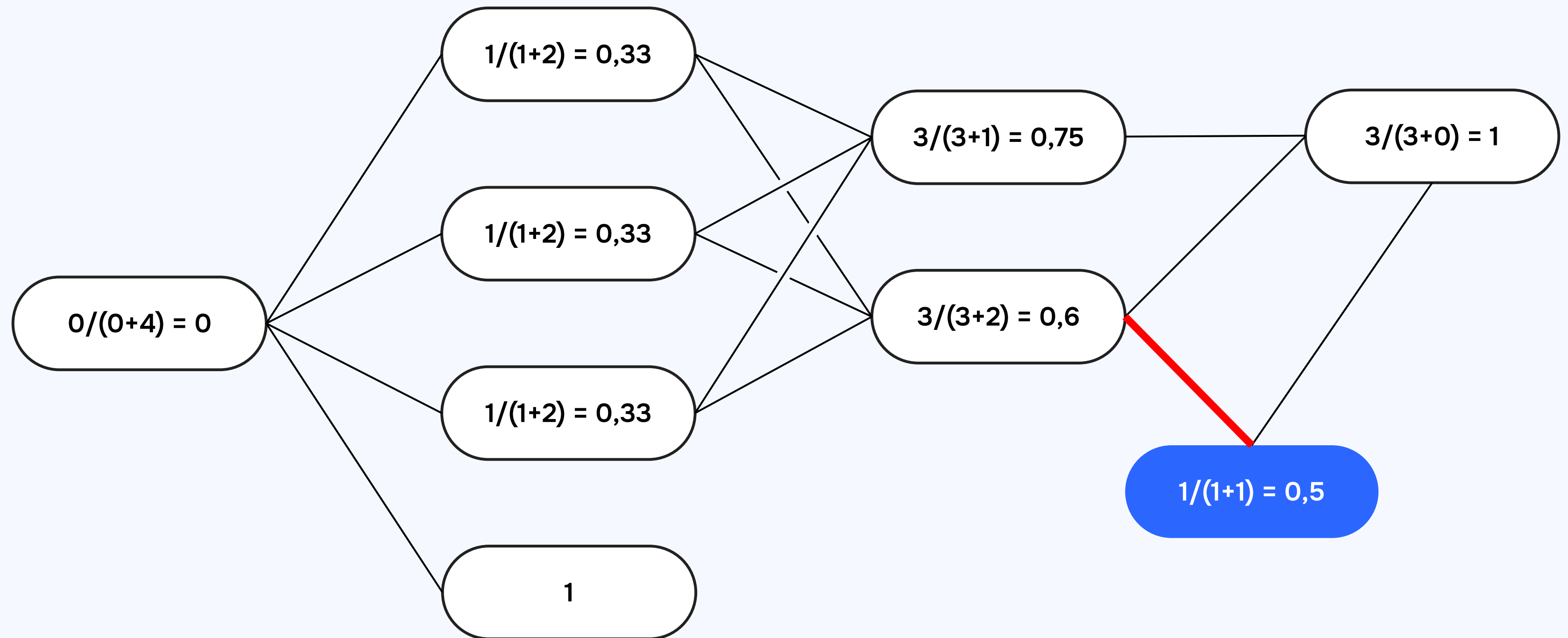


3 способа завязать узел

3 способа завязать узел

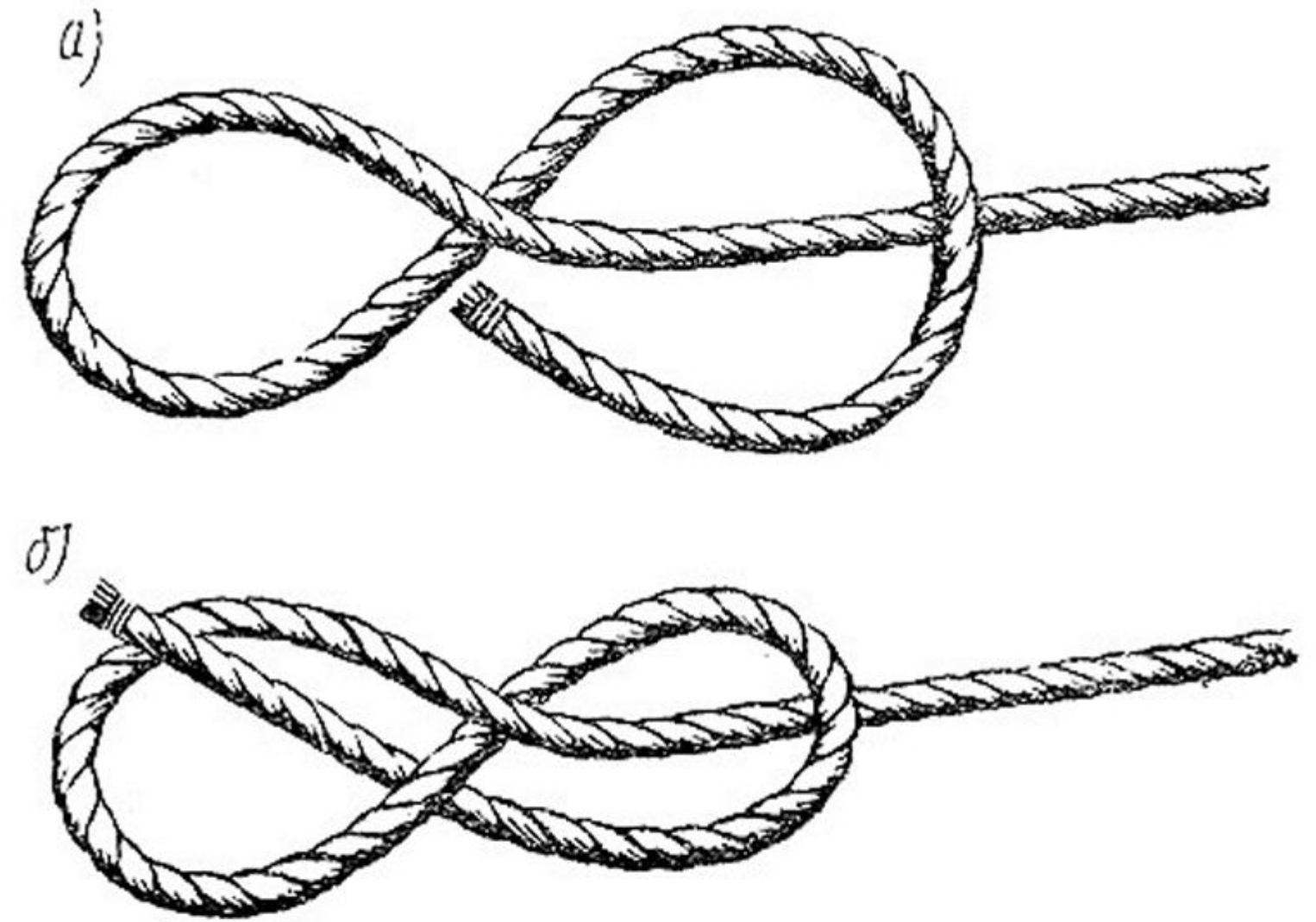
- Управлять соседями по уровню

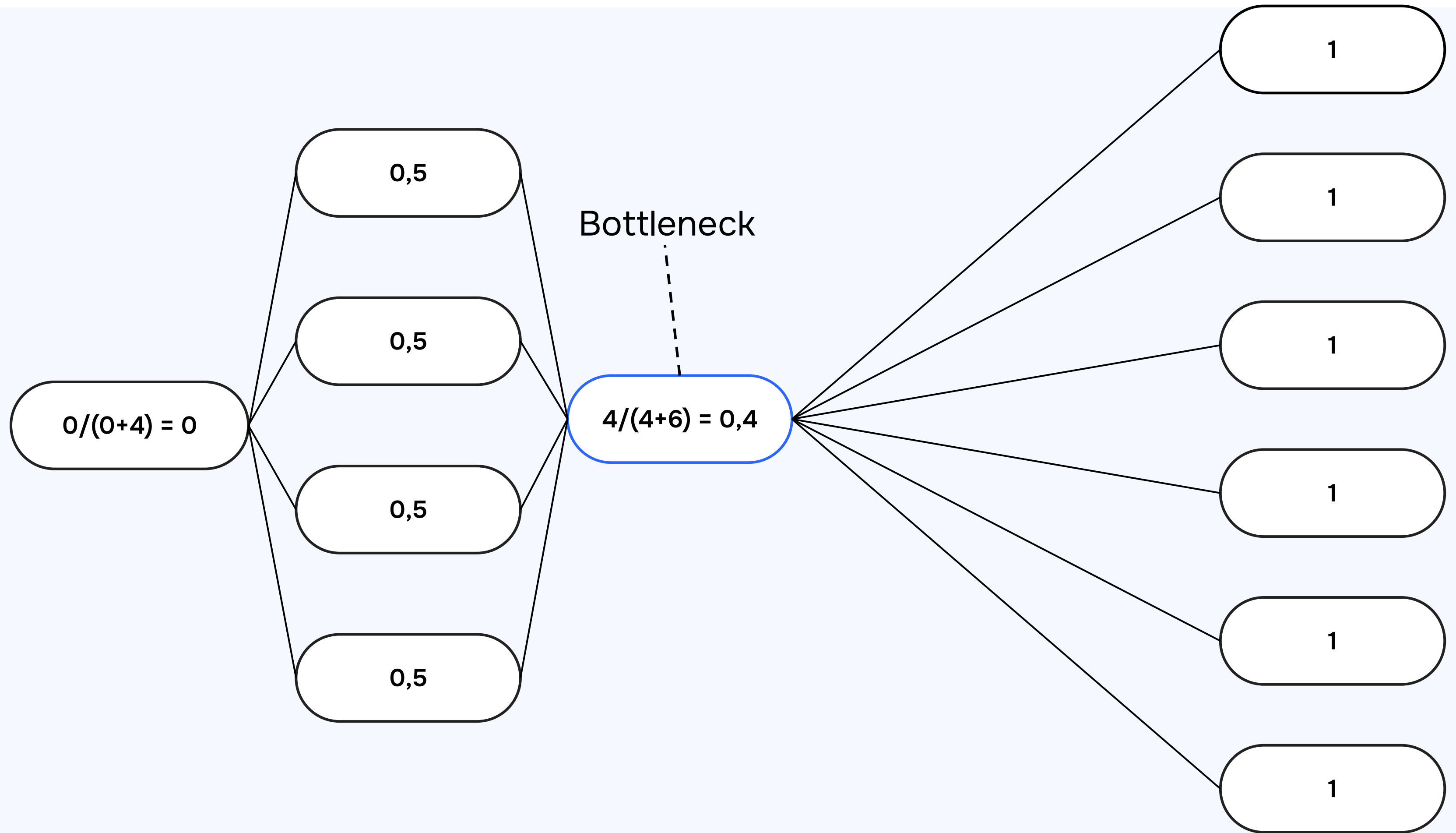




3 способа завязать узел

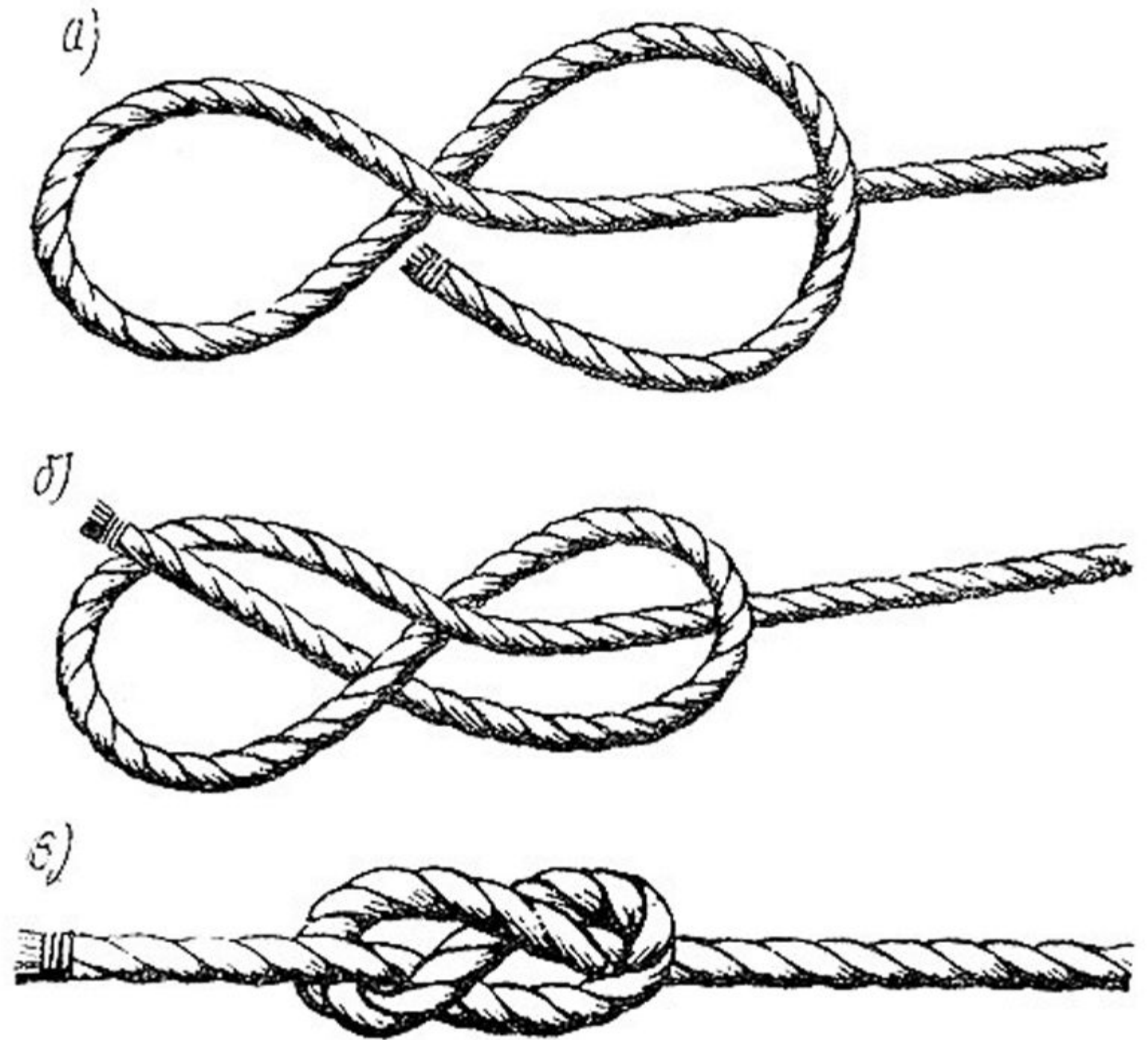
- Управлять соседями по уровню
- Воткнуть хотспот, перегруженный ответственностью

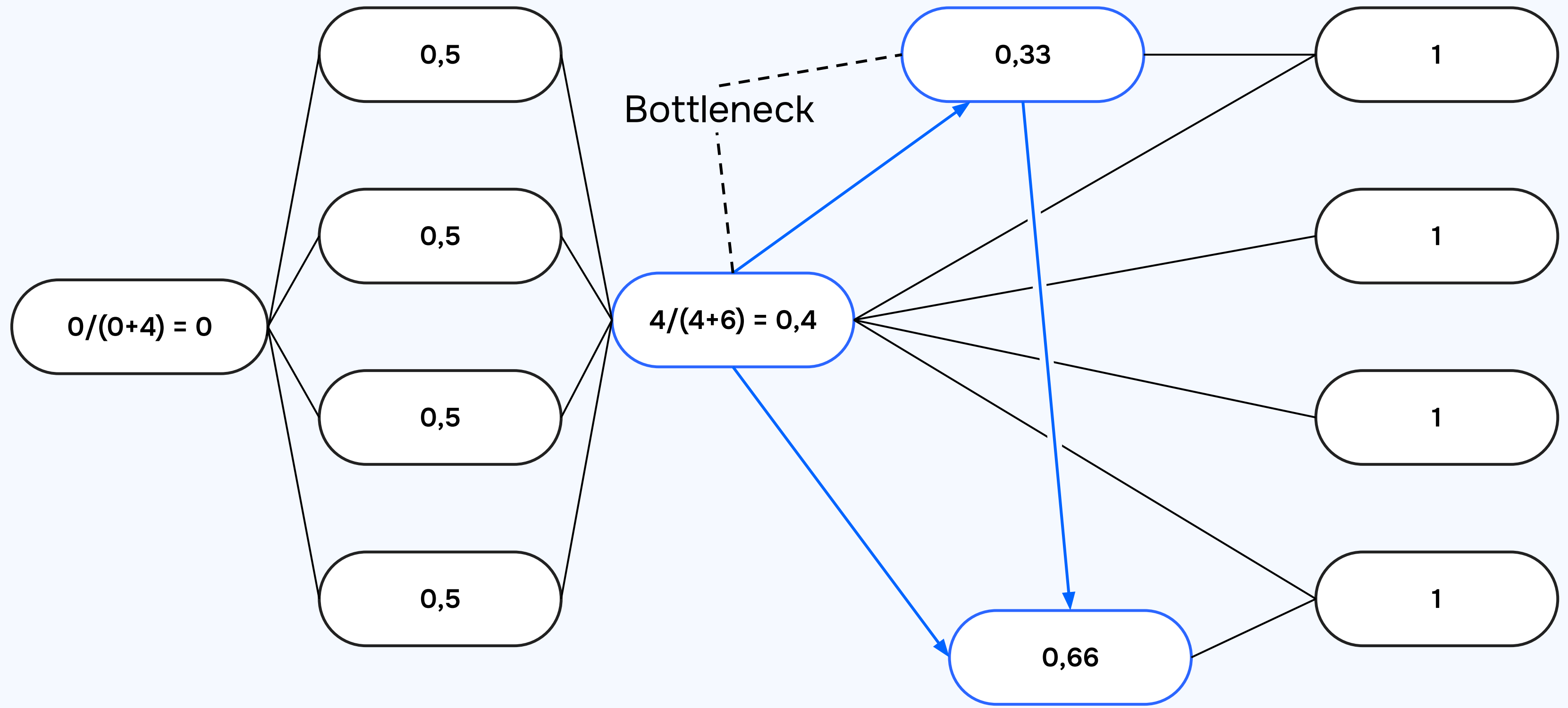




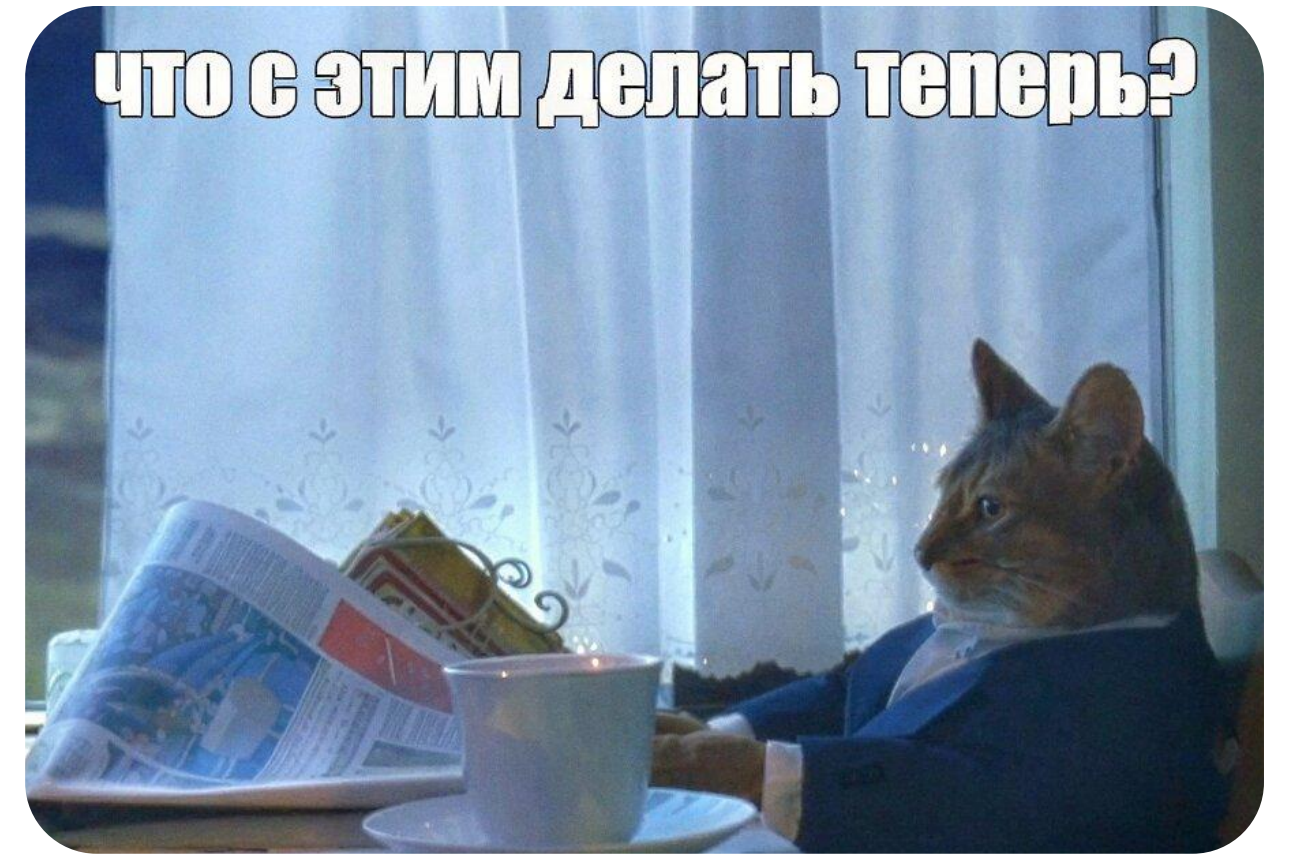
3 способа завязать узел

- Управлять соседями по уровню
- Воткнуть хотспот, перегруженный ответственностью
- Устроить циклическое замыкание



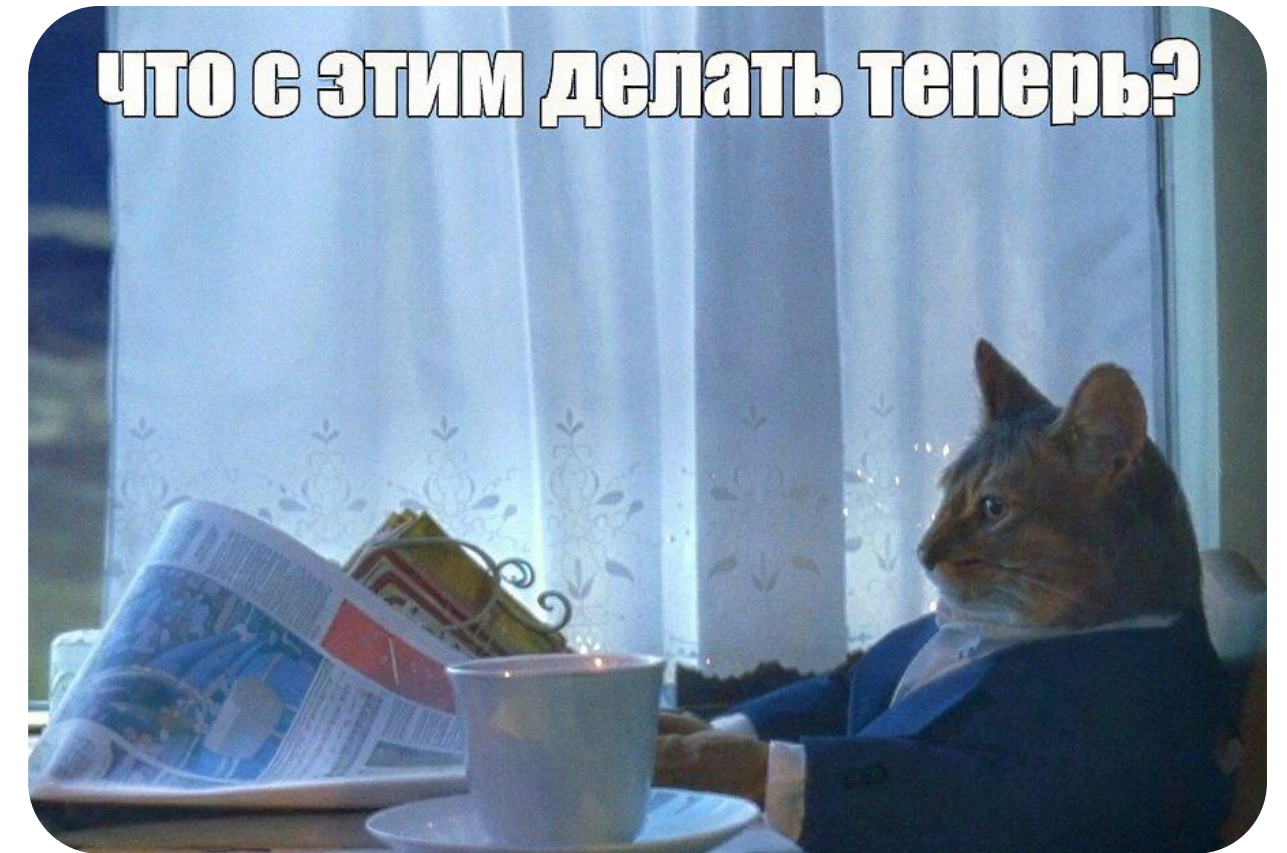


Контролировать связи



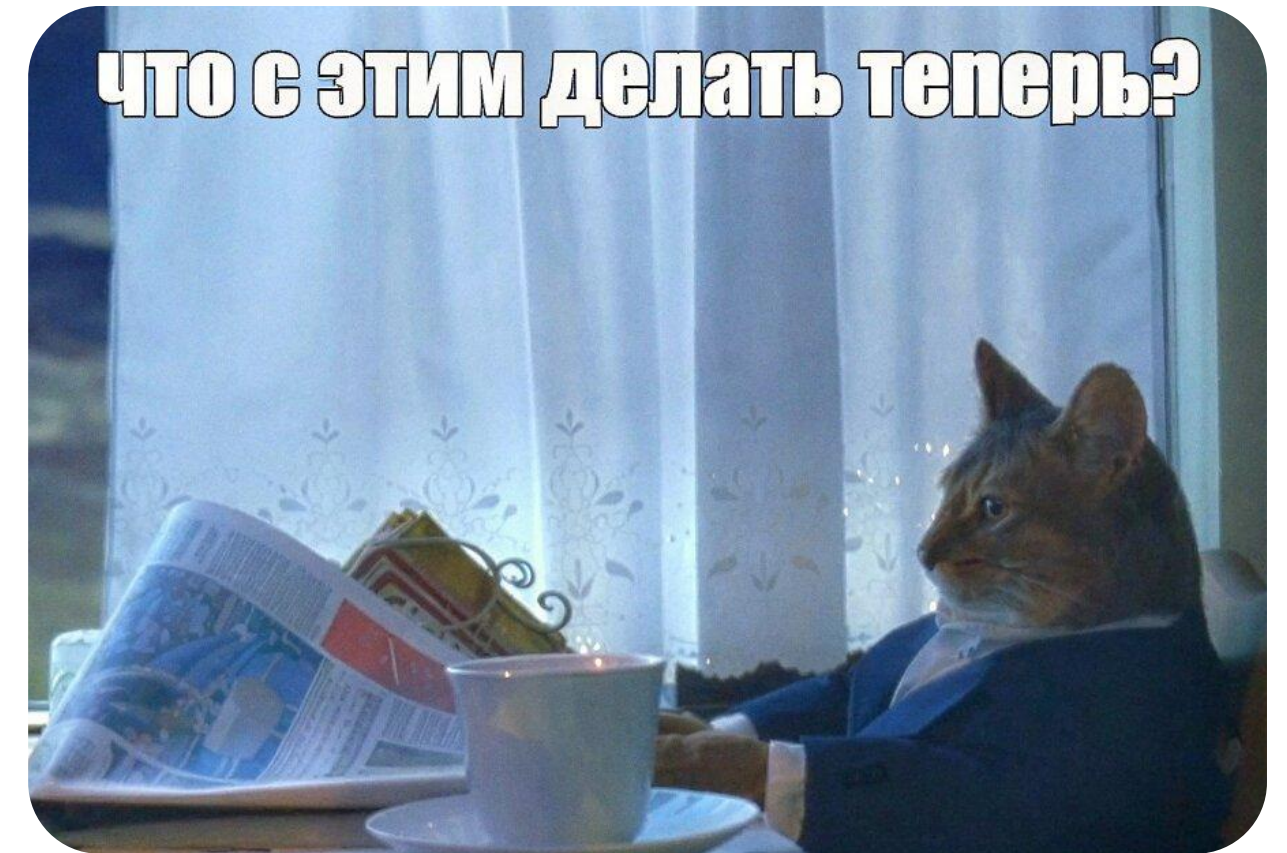
Контролировать связи

- ✓ Соблюдать иерархию



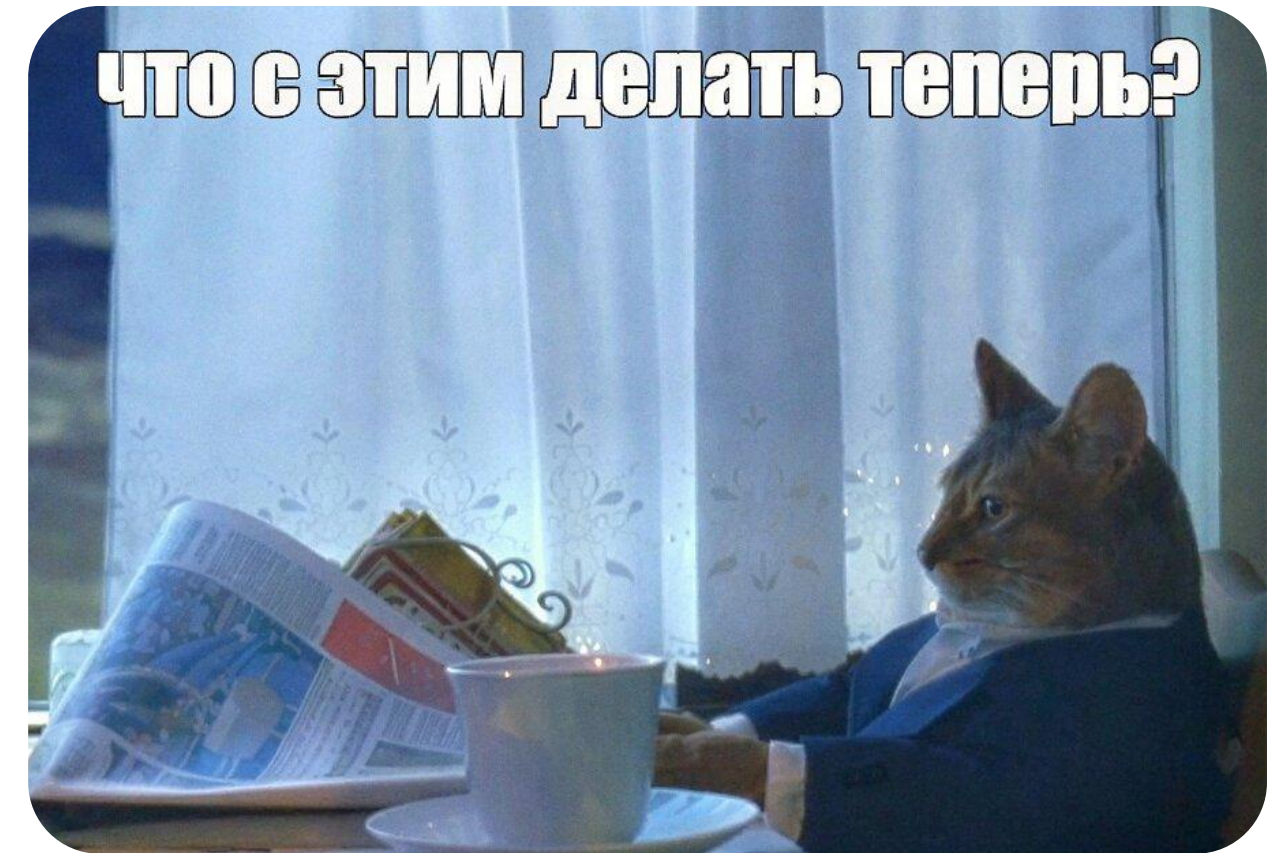
Контролировать связи

- ✓ Соблюдать иерархию
- ✓ Зависимости должны быть стабильнее, чем сам компонент



Контролировать связи

- ✓ Соблюдать иерархию
- ✓ Зависимости должны быть стабильнее, чем сам компонент
- ✓ Не раздувать компонентеты.
Объединить проще, чем распилить



История 7. Долбанёт? Вероятно, нет!



Одно из требований к нашей системе —
аптайм должен быть 99,99 %.

ОК, сделаем и потом дотюним.

Что за этим скрывается?



10 компонентов, 10 % вероятность поломки

10 %

10 %

10 %

10 %

10 %

10 %

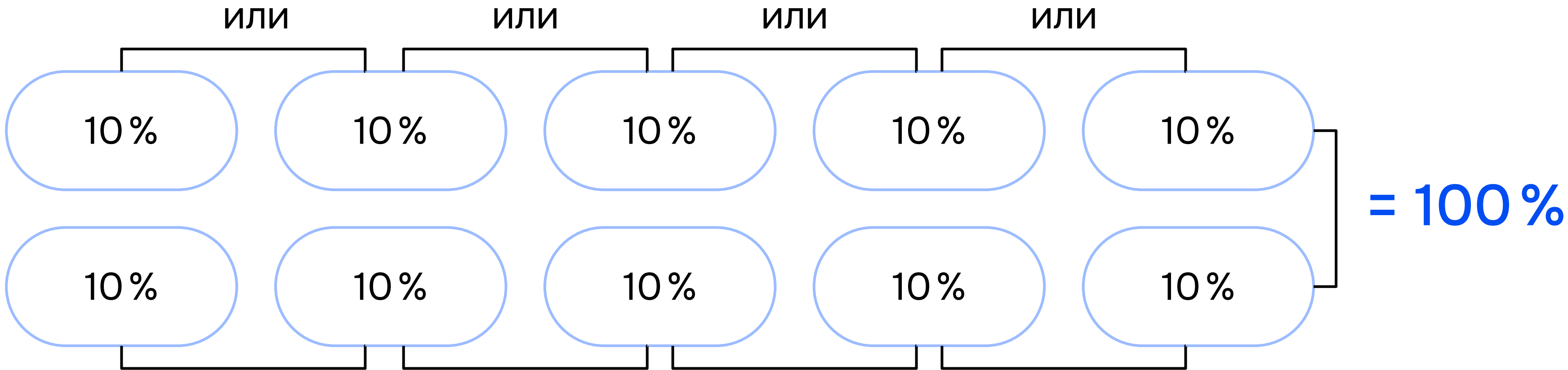
10 %

10 %

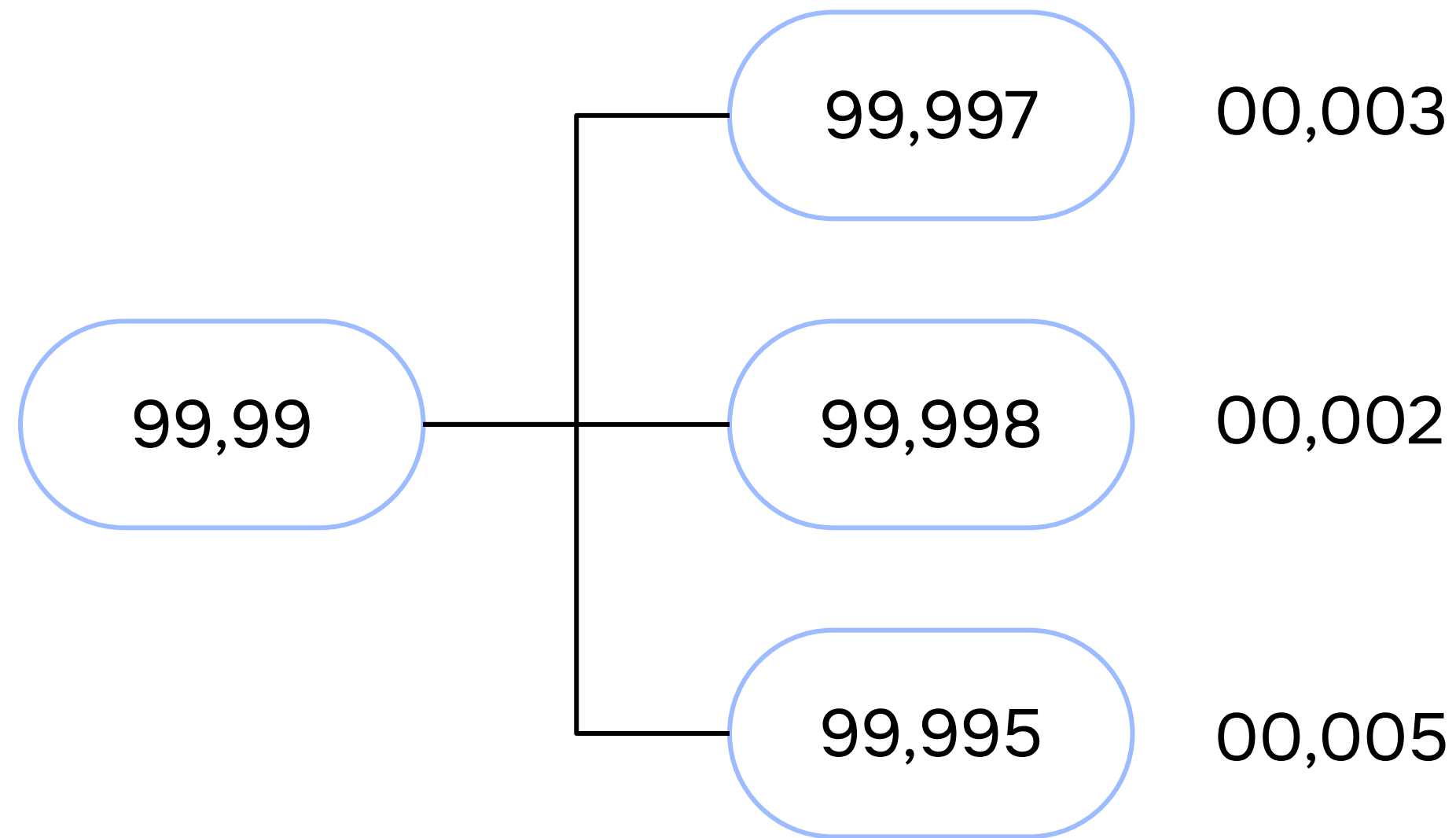
10 %

10 %

Ошибки неизбежны



99,99 uptime системы != 99,99 uptime компонентов



00,01%
downtime
на всех!

**Проектировать
доступность с конца**

Проектировать доступность с конца

☒ Нужно ли это такой ценой?

Проектировать доступность с конца

- ✓ Нужно ли это такой ценой?
- ✓ Запас прочности

Проектировать доступность с конца

- ✓ Нужно ли это такой ценой?
- ✓ Запас прочности
- ✓ “Мягкие” связи

Проектировать доступность с конца

- ✓ Нужно ли это такой ценой?
- ✓ Запас прочности
- ✓ “Мягкие” связи



Вот и сказочкам конец, а в
финале...

Почти получилось



Почти получилось

Игнорировать мелочи - перепрыгнуть
лужу на 98%



Почти получилось



Игнорировать мелочи - перепрыгнуть
лужу на 98%

+ В целом, совпали приоритеты

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст
- + Слегка сгладили углы

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст
- + Слегка сгладили углы
- + Немного переборщили с полномочиями

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст
- + Слегка сгладили углы
- + Немного переборщили с полномочиями
- + Вроде оценили ресурсы на поддержку

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст
- + Слегка сгладили углы
- + Немного переборщили с полномочиями
- + Вроде оценили ресурсы на поддержку
- + На глаз создавали связи

Почти получилось



Игнорировать мелочи - перепрыгнуть лужу на 98%

- + В целом, совпали приоритеты
- + Чуть-чуть размыли контекст
- + Слегка сгладили углы
- + Немного переборщили с полномочиями
- + Вроде оценили ресурсы на поддержку
- + На глаз создавали связи
- + Потом **как-нибудь** выкручивались

Губят не эпичные вещи, а
мелочи, которые мы
пропускаем каждый день.

Полезно возвращаться назад и
переспрашивать "а почему мы
так делаем?"

Deckhouse Kubernetes Platform Community Edition



🎧 Готовая Open Source-платформа
60+ компонентов закрывают Day-2-потребности

🤖 Автоматизация и минимальный Human-in-the-Loop
Установка, обновления, изоляция, скейлинг, хилинг

⚙️ Без ограничений
Тонкая настройка. Open core

Оцените доклад

Павла Богданова

 @deckhouse_news

 github.com/deckhouse

